



Campus Coin

Project Report

Smart Spending, Student Style - a budget and expense tracker for students

End-to-End Web Solutions · Theme: NextGen BudgetBee

Submitted by	Hamza Haroon
Student ID	-----
Faculty	-----
Centre	Aptech Metro Star Gate, Karachi
Stack	MongoDB, Express.js, React, Node.js (MERN)
Live site	https://campuscoin-sable.vercel.app
Source code	https://github.com/hamzaharoonk07-cloud/campuscoin
Version	1.0 · September 2026

Declaration

I hereby declare that this project, **Campus Coin – Smart Spending, Student Style**, submitted for the End-to-End Web Solutions category (theme: NextGen BudgetBee), is my own work. It was designed and built by me as part of my studies at Aptech, and it has not been submitted elsewhere for any other award. All external libraries, assets and tools used are acknowledged in this report.

Signature

Date

Acknowledgement

I would like to thank my faculty at Aptech for their guidance throughout this project, and for the Software Requirements Specification that defined its goals. I am also grateful to my classmates, who tested Campus Coin with their own spending and pointed out what a student actually needs from a budget app, and to my family for their support.

Thanks are also due to the open-source projects this application is built on - React, Node.js, Express, MongoDB, Tesseract and others - and to the designers whose work on Dribbble inspired the look of the interface.

Abstract

Students manage money that arrives irregularly – an allowance, part-time work, a scholarship, a gift – and spend it in many small amounts that are easy to lose track of. Existing finance apps are built for salaried adults and bank feeds, and do not fit this pattern.

Campus Coin is a responsive MERN-stack web application (MongoDB, Express.js, React, Node.js) that lets a student record income and expenses in seconds, sorted into student-relevant categories. A built-in assistant suggests the category as the student types and learns from corrections; receipts can be photographed and read automatically. The student sets a monthly budget per category and is alerted at 80% and 100%. Dashboards, a money calendar, monthly and six-month reports, a monthly insight that flags unusual growth, and saving tips ranked by the money they would save are all calculated from the student's own history. An administrator manages default categories, announcements and user accounts.

The application is secured with hashed passwords, versioned sessions, logout and email-link password changes, is deployed publicly on Vercel with MongoDB Atlas, and is verified by 65 automated end-to-end tests covering every functional requirement.

Keywords: budget tracker, expense categorisation, MERN stack, personal finance, students, React, Node.js, MongoDB

Contents

	Declaration, Acknowledgement, Abstract	2
1	Problem Definition	8
2	Proposed Solution and Scope	9
3	Design Specifications	10
4	System Diagrams	13
	4.1 Use-case diagram	13
	4.2 System architecture	14
	4.3 Sitemap	15
	4.4 Flowcharts	16
	4.5 Data flow diagrams	22
	4.6 Sequence diagram	24
5	Database Design	25
6	Functional Requirements and How They Are Met	29
7	Non-Functional Requirements	31
8	User Interface	32
9	Test Data and Testing	47
10	Project Installation Instructions	51
11	User Credentials	53
12	Acknowledgement of AI Tools	54
13	Conclusion and Future Work	55
14	Glossary and References	56

List of Figures

1	Use-case diagram	13
2	Three-tier system architecture	14
3	Sitemap and navigation structure	15
4	Flowchart - student registration	16
5	Flowchart - student sign-in with logout	17
6	Flowchart - adding a transaction with AI categorisation	18
7	Flowchart - password recovery and change by tokenised link	19
8	Flowchart - generating the monthly insight	20
9	Flowchart - administrator control panel	21
10	Data flow diagram, level 0 (context diagram)	22
11	Data flow diagram, level 1	23
12	Sequence diagram - adding a transaction with an AI-suggested category	24
13	Entity-relationship diagram (MongoDB collections)	25
14	Landing page	32
15	Student sign-in with the demo account button	33
16	Dashboard: greeting, money in and out, cash flow, where it went, top category, budget vs actual	34
17	Dashboard in dark mode	35
18	Transactions with category pictures and merchant logos	36
19	Money calendar: each day's spending and the selected day's transactions	37
20	Budgets with live progress bars	37
21	Monthly report	38

22	Monthly insights	39
23	Saving tips ranked by impact	40
24	Categories: manage own categories and the defaults	41
25	Coin, the chat assistant	42
26	Notifications panel	42
27	Account settings	43
28	Admin control panel: usage statistics	44
29	Admin: student accounts	45
30	Dashboard on a phone	46
31	Welcome email	46

1 Problem Definition

Why a student-specific budget tracker is needed.

College and university students receive money from several irregular sources: a monthly allowance from home, part-time or tutoring work, scholarship instalments and occasional gifts such as Eidi. Very few of them know where that money goes by the end of the month. The personal-finance apps that exist are designed for salaried adults with a fixed pay day and a bank feed; they are often subscription-based, ask for bank details, and use categories that do not match a student's life.

A student's spending is made of small, frequent purchases – chai at the canteen, a rickshaw to campus, photocopies and printing, a shared Netflix plan – together with a few large, fixed costs such as hostel rent. These small items are easy to forget, and they are exactly the ones that add up. Without a simple record, a student usually discovers a problem only when the money has already run out.

1.1 The problem in short

- Students have no quick, free way to record income and expenses in categories that fit student life.
- They cannot see which category is growing, or compare this month with their own usual spending.
- Generic advice ("spend less") is not useful; advice needs to come from the student's own numbers.
- Linking a bank account is neither possible nor desirable for most students.

1.2 Objectives

- Let a student record a transaction in a few seconds, with the category suggested automatically.
- Show the month at a glance: balance, top category, budget against actual spending.
- Set a budget per category and warn the student in time, at 80% and at 100%.
- Produce monthly reports, a plain-language monthly insight and ranked saving tips from the student's own history.
- Give an administrator control over default categories, announcements and user accounts.
- Work on phones and computers, in light and dark mode, without any bank connection.

2

Proposed Solution and Scope

What Campus Coin does, and where its boundaries are.

Campus Coin is a responsive full-stack web application. A student registers, logs in and immediately starts recording income (allowance, part-time job, scholarship, gift, other income) and expenses (food, transport, hostel/rent, academics, subscriptions, entertainment, miscellaneous). Every transaction is stored in a central MongoDB database, which powers the dashboard, category-wise and six-month reports, the calendar, budgets and alerts, the saving-tips engine and the monthly insights.

An intelligent assistant runs through the application: it suggests a category while the student types a description and learns from corrections, reads the amount, shop and date from a photographed receipt, writes a monthly summary that flags unusual growth, and answers questions in a chat window ("How much did I spend on food?") from the student's own data.

2.1 Scope

In scope	Out of scope (by the SRS constraints)
Manual entry, CSV import and receipt scanning of transactions	Linking to real bank accounts or wallets
Budgets, alerts, reports, insights, saving tips, calendar, chat assistant	Real payments or money transfers
Student and administrator accounts with secure sessions and email	Certified financial advice (all advice is marked as a suggestion)
Responsive layout, dark mode, text-size control	Native mobile apps (the web app works on phones instead)

2.2 Constraints followed

- No bank integration: all data is entered by the student, imported from a CSV file or read from a receipt photo.
- Every suggestion from the categoriser, insight writer and tips engine can be reviewed and overridden, and is labelled as advice, not a decision.
- No ready-made website template or UI component library was used. The design system, layout and every chart are written in CSS and SVG as part of the project.

3

Design Specifications

Technology, modules, security and interface design.

3.1 Technology

Layer	Technology	Why it was chosen
Front end	React 18, React Router, Vite	Component-based screens, fast builds, one-page navigation without reloads
Styling and charts	Hand-written CSS design tokens; SVG charts	Full control of the design; charts that stay sharp on any screen; no template
Back end	Node.js, Express 5	One language (JavaScript) across the stack; simple REST routes
Database	MongoDB with Mongoose	Flexible documents for transactions, insights and recurring rules; free Atlas hosting
Security	bcrypt, JSON Web Tokens, crypto tokens	Salted password hashes, signed sessions, one-time reset links
Receipt reading	Tesseract.js (in the browser)	OCR without uploading the photo or paying for an API
Email	Nodemailer with Gmail SMTP	Password links, welcome and summary emails
Optional AI wording	Anthropic Claude API	Rewrites already-computed monthly figures in warmer language
Hosting	Vercel (serverless API + static client), MongoDB Atlas	Free, always-on public deployment with HTTPS

3.2 Modules

Module	Responsibility
Authentication	Registration, student and separate admin sign-in, logout after 5 wrong passwords, sessions that end on password change, "sign out everywhere", password recovery and change by emailed link
Profile and settings	Name, study level (Matric to PhD), institution, monthly allowance, savings goal, currency, photo, theme and text size
Categories	Five default income and seven default expense categories; students add, edit and delete their own ("Manage own categories")

Module	Responsibility
Transactions	Quick add, recurring entries, edit and delete, search and filters, CSV import with batch suggestions, CSV export, receipt photos
Categoriser (AI)	Suggests a category from the description using the student's own word-to-category history; learns from every correction
Anomaly detection	Flags unusually large amounts and likely duplicates
Budgets and alerts	A monthly cap per category, live progress bars, in-app notifications at 80% and 100%, copy to next month
Dashboard and calendar	Greeting, balance, top category, budget vs actual, cash flow, spending donut; a calendar of every day's money
Reports	Category-wise month, six-month income vs expense, daily and weekly summaries, filters, forecast, PDF and image export
Insights	Monthly narrative, month score, category changes, spending habits, history, bookmarks, email sharing
Saving tips	Rules that compare the month with the student's own averages, ranked by money saved; pin and dismiss
Chat assistant (Coin)	Answers money questions from the same figures the reports use
Administration	Usage statistics, student accounts (view, disable, reset, remove), default categories, announcements and tip templates

3.3 Security design

- Passwords are stored only as salted bcrypt hashes (cost 10) and must be at least 8 characters with letters and numbers, not a common password and not built from the user's name or email.
- Each session token carries the account's session version; changing or resetting a password raises it, which ends every other session immediately.
- Five wrong passwords in a row lock sign-in for 15 minutes. Disabled accounts lose access at once.
- A password can only change through a one-time link, valid for one hour, emailed to the account's own address. Only the SHA-256 hash of the link's token is stored.
- Every data query is filtered by the signed-in user, so a student can only ever read their own transactions. Admin routes check the admin role on every request.

3.4 User-interface design

The interface follows a black, white and blue scheme: a slim icon sidebar that opens on hover, one white content panel, black pill buttons and a single blue (#5B91FF) for figures and money coming in. Text is set in Plus Jakarta Sans. Categories use full-colour pictures, and merchants such as Netflix, Spotify and foodpanda show their own logos. On phones the sidebar becomes a bottom tab bar and cards stack in one column. A light and dark theme, adjustable text size and breadcrumbs are provided for accessibility.

4

System Diagrams

How the system is structured and how data and control flow through it.

4.1 Use-case diagram

Two kinds of user work with Campus Coin. The student records and reviews their own money; the administrator looks after the shared parts of the system. The diagram shows what each can do.

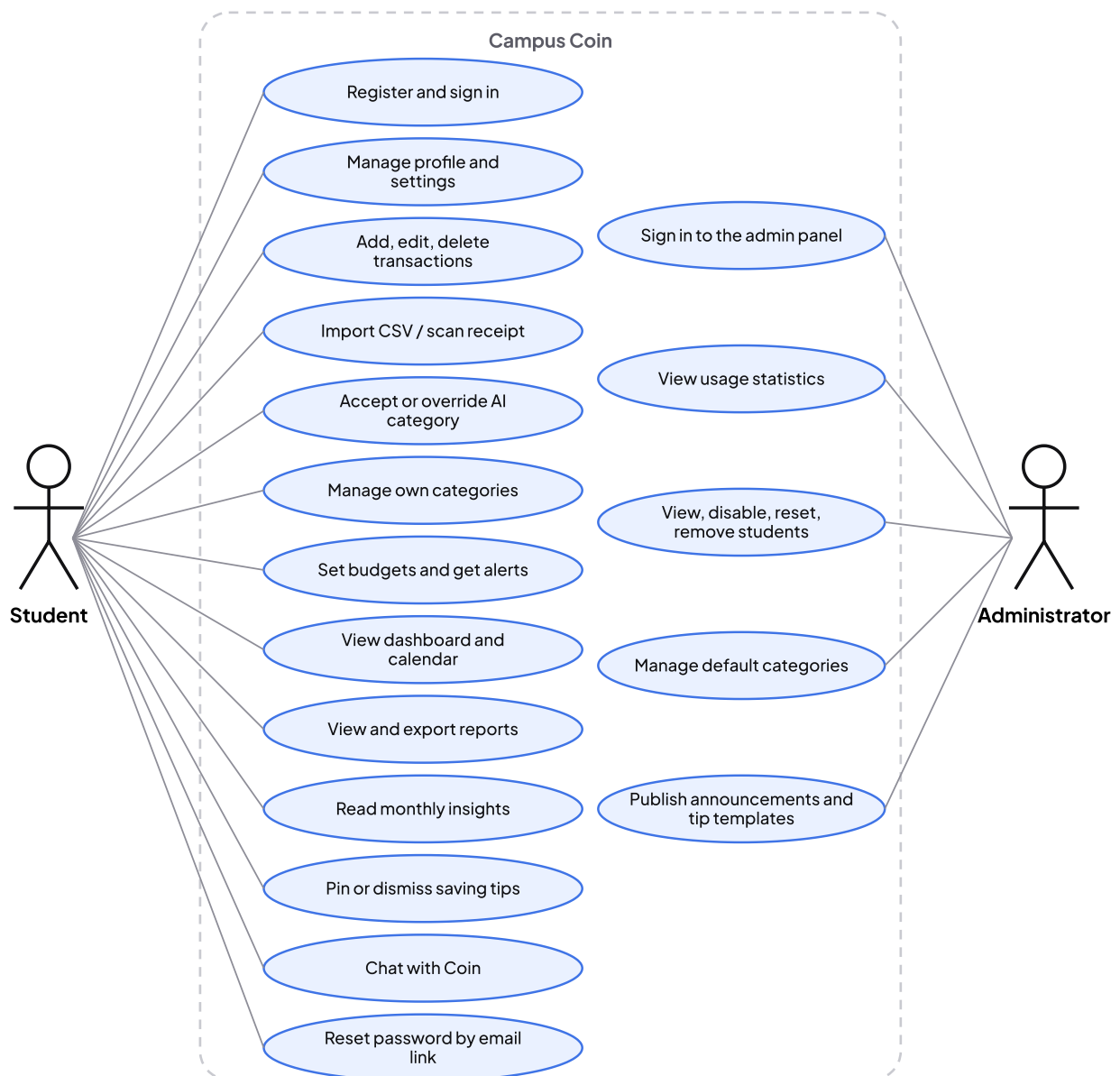
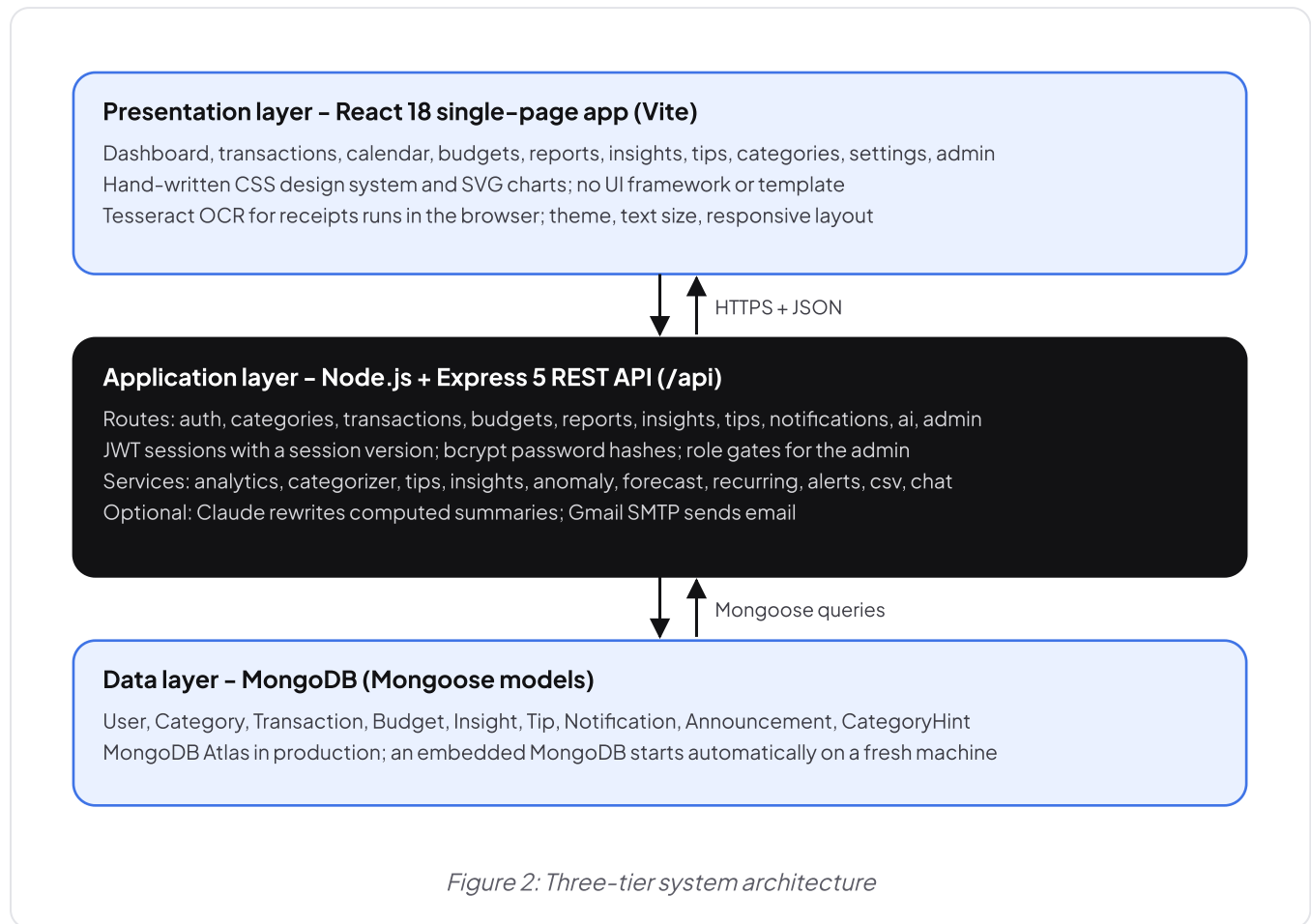


Figure 1: Use-case diagram

4.2 System architecture



The application follows the three-tier design described in the SRS. The React client never talks to the database directly; it calls the Express API over HTTPS, and every figure shown on screen is calculated by the server's service layer from the database.

4.3 Sitemap

Public pages lead into two separate areas. Students and the administrator sign in at different addresses, and each area is protected on the server.

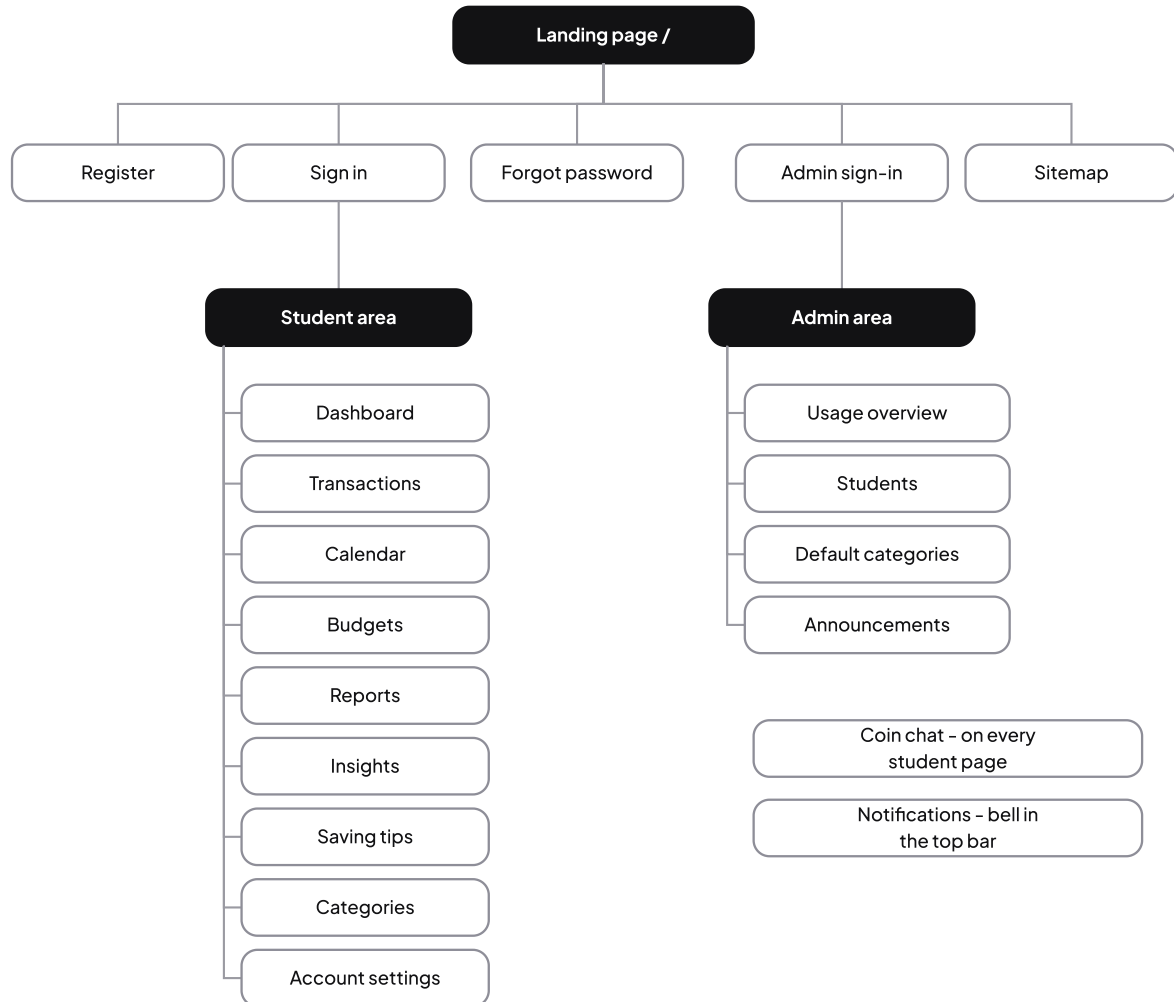


Figure 3: Sitemap and navigation structure

4.4 Flowcharts

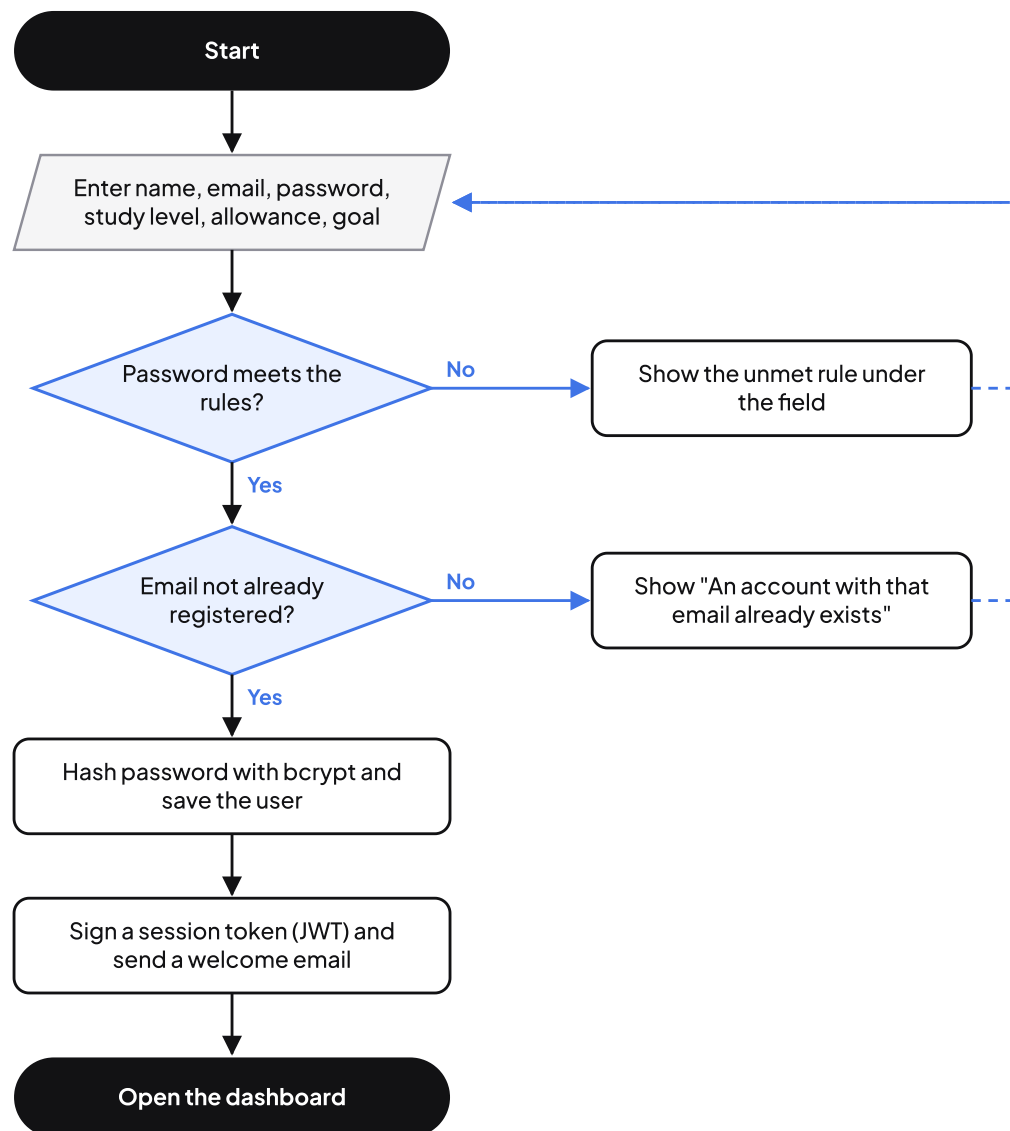


Figure 4: Flowchart – student registration

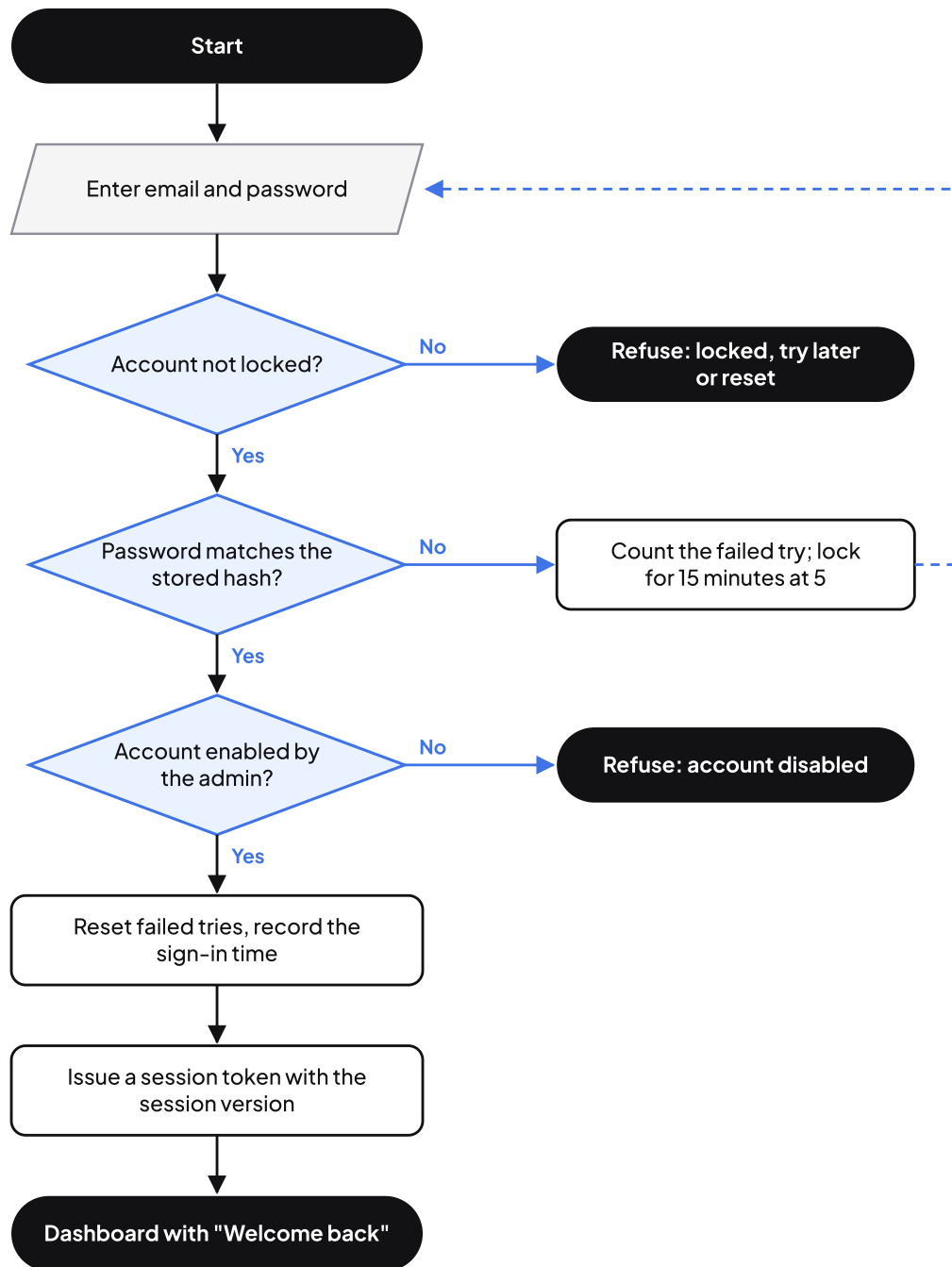


Figure 5: Flowchart – student sign-in with lockout

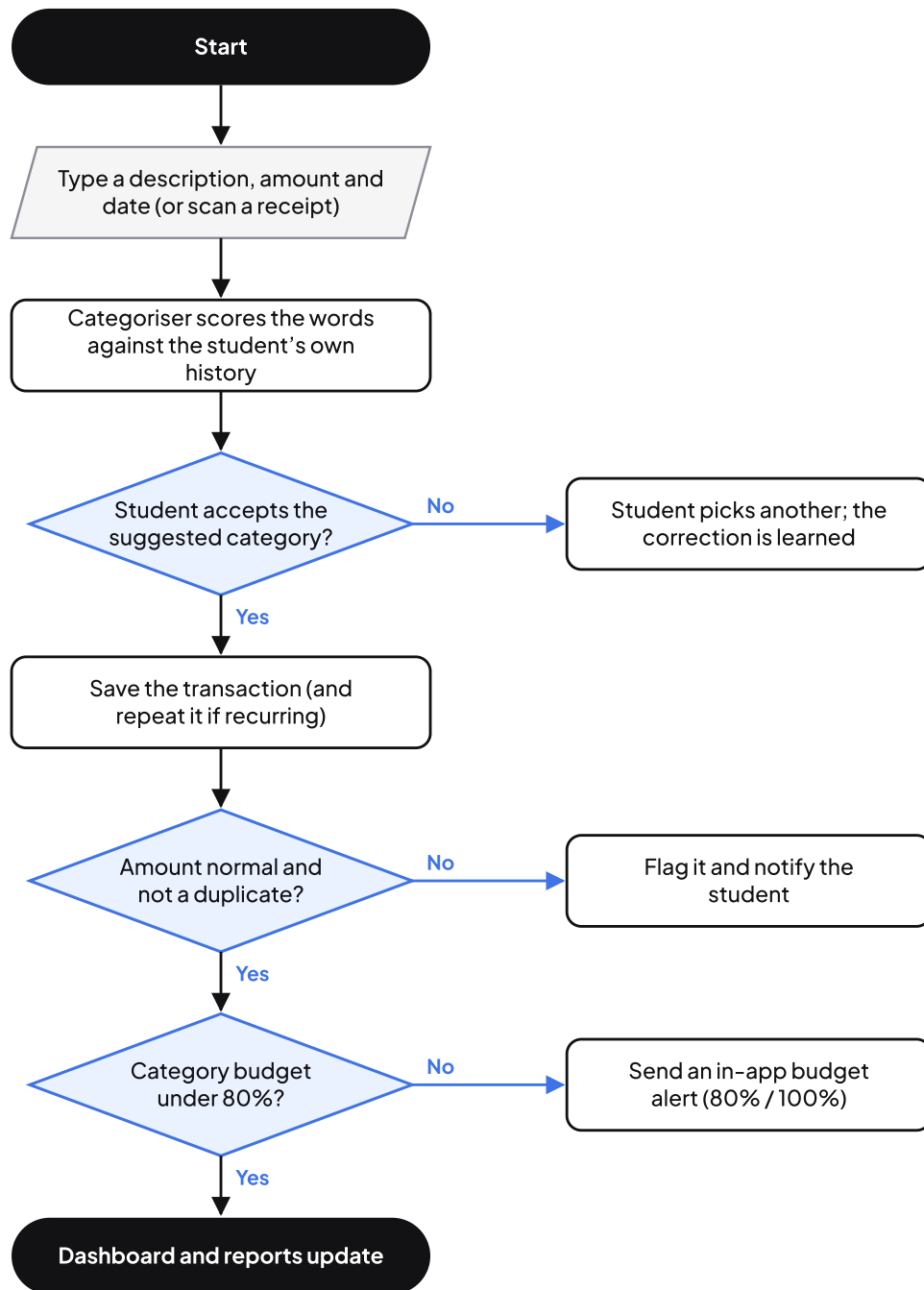


Figure 6: Flowchart - adding a transaction with AI categorisation

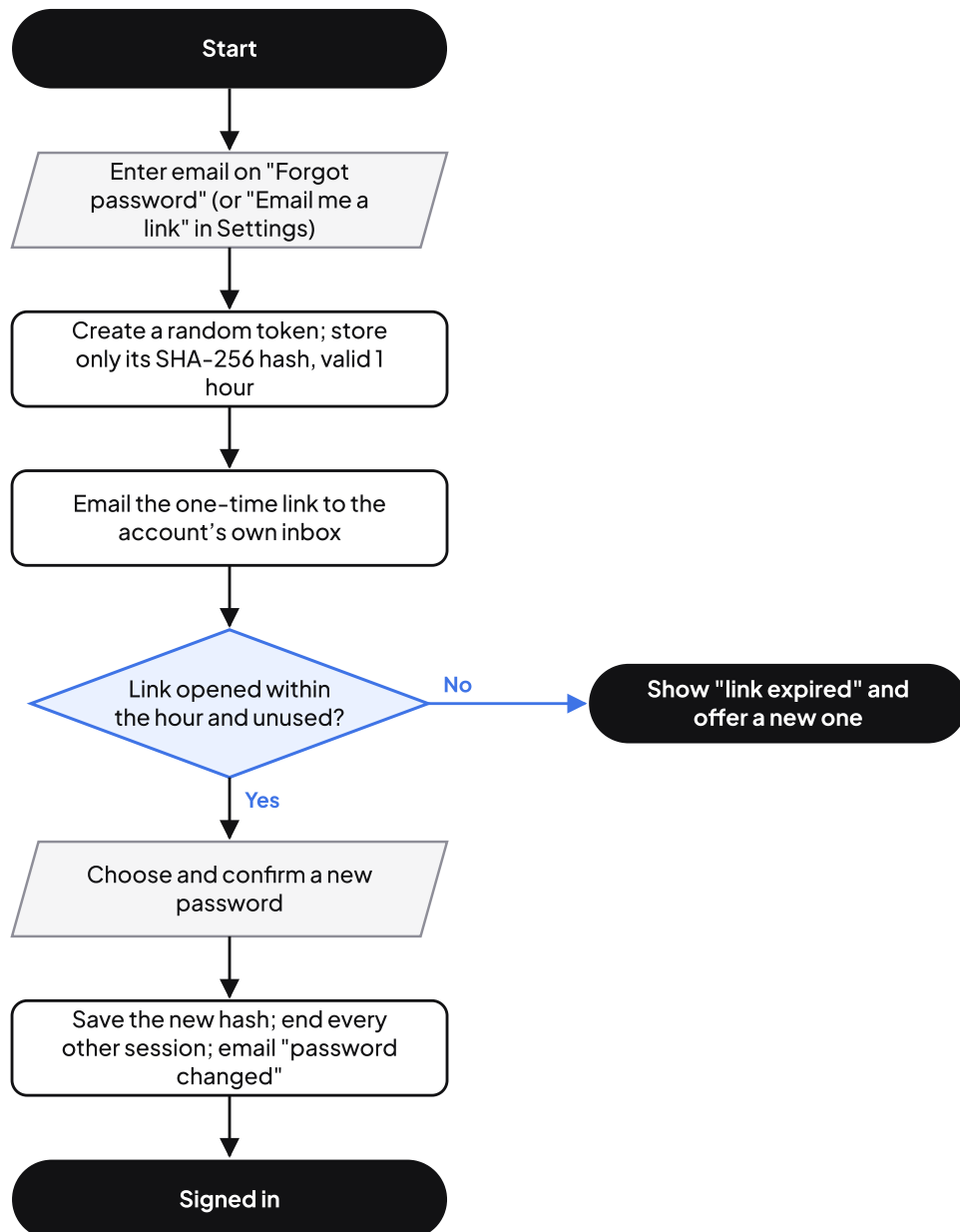


Figure 7: Flowchart - password recovery and change by tokenised link

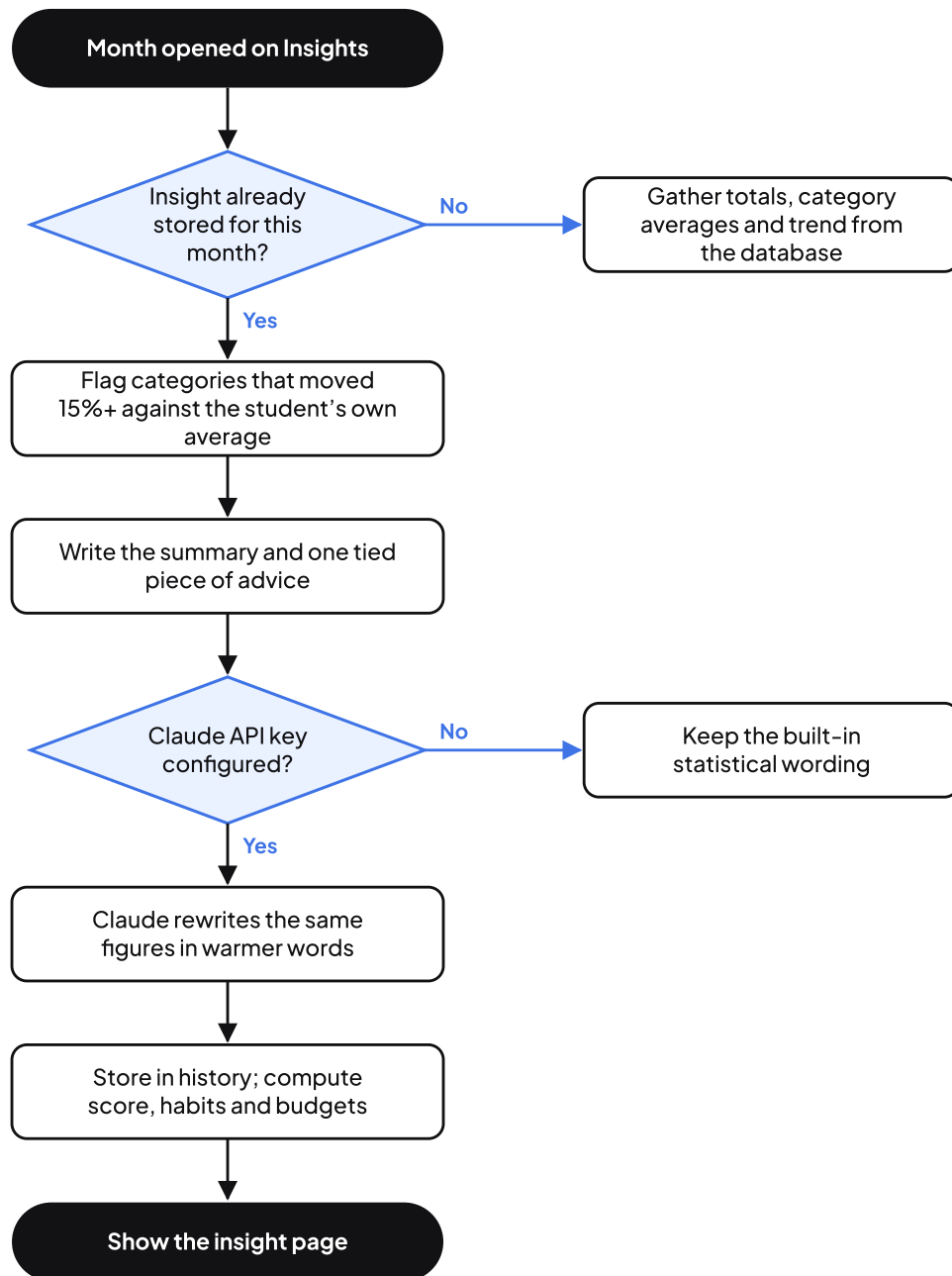


Figure 8: Flowchart - generating the monthly insight

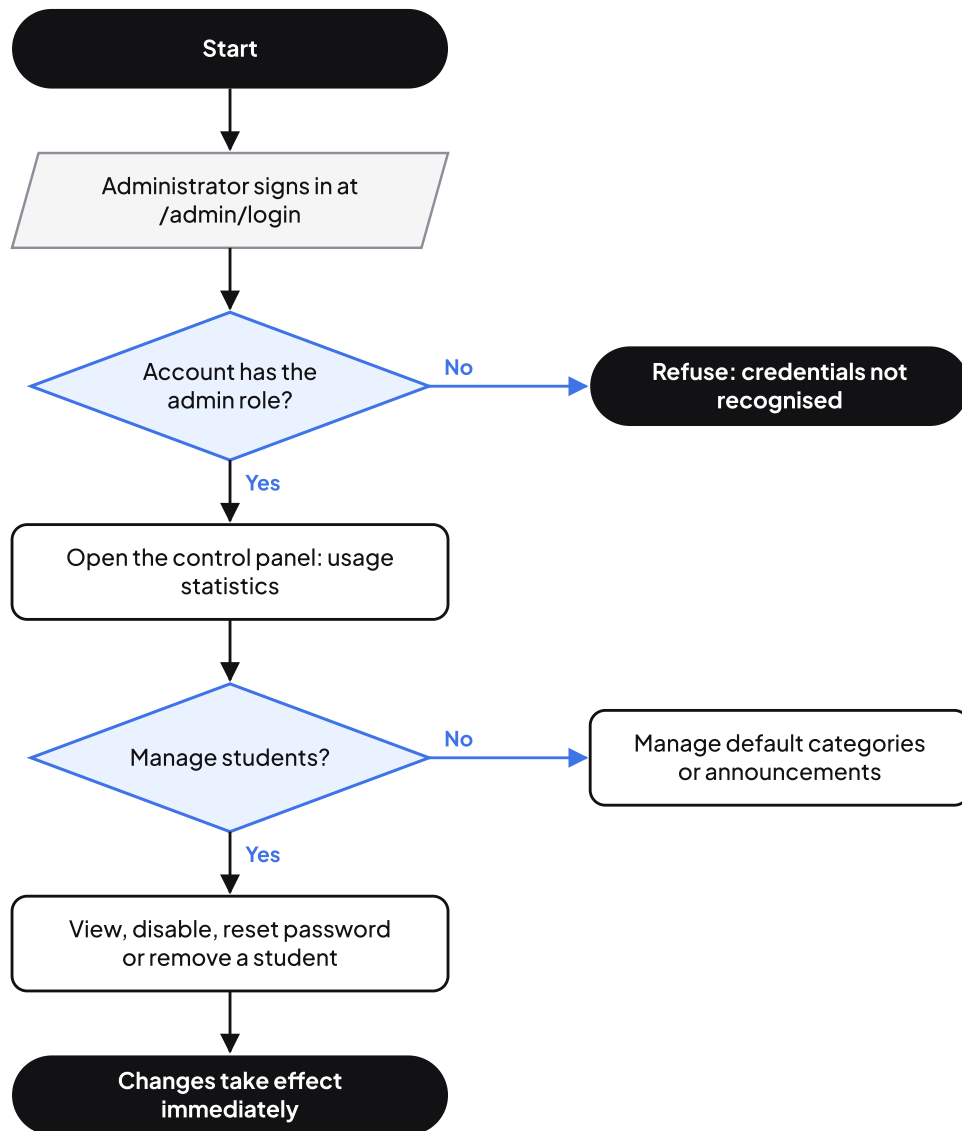
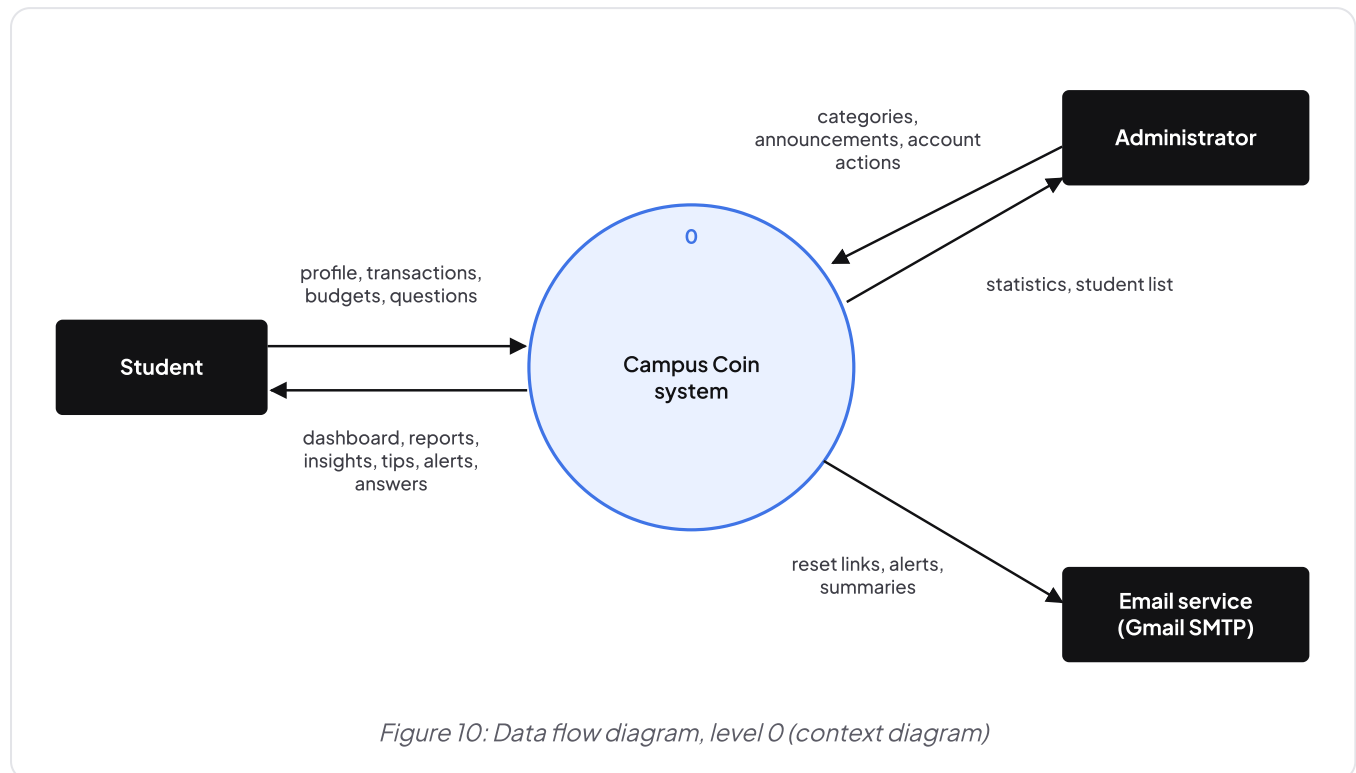


Figure 9: Flowchart - administrator control panel

4.5 Data flow diagrams



At the context level, the student sends profile details, transactions, budgets and questions into the system and receives the dashboard, reports, insights, tips, alerts and answers. The administrator manages categories, announcements and accounts and receives usage statistics. The email service delivers password links, alerts and summaries.

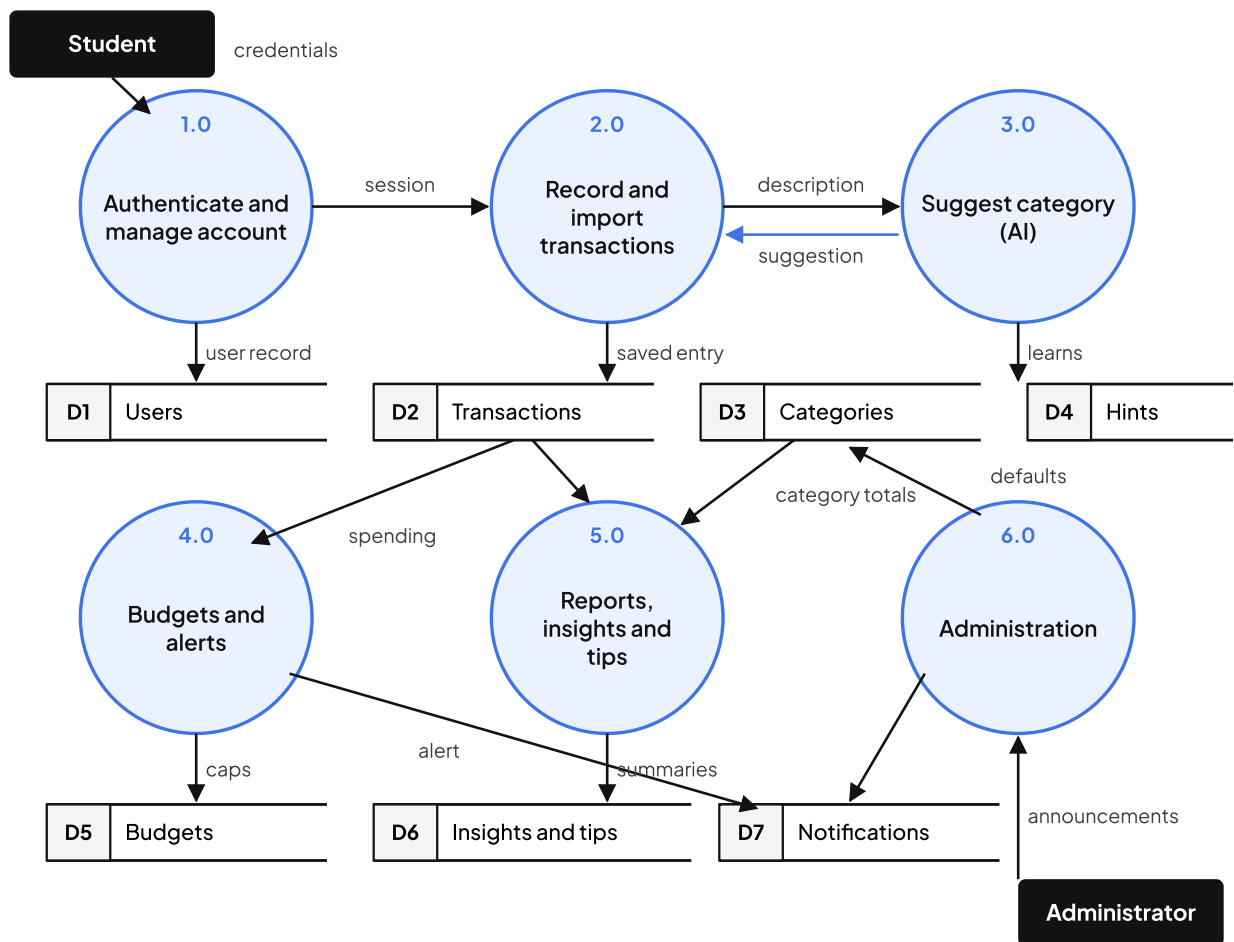
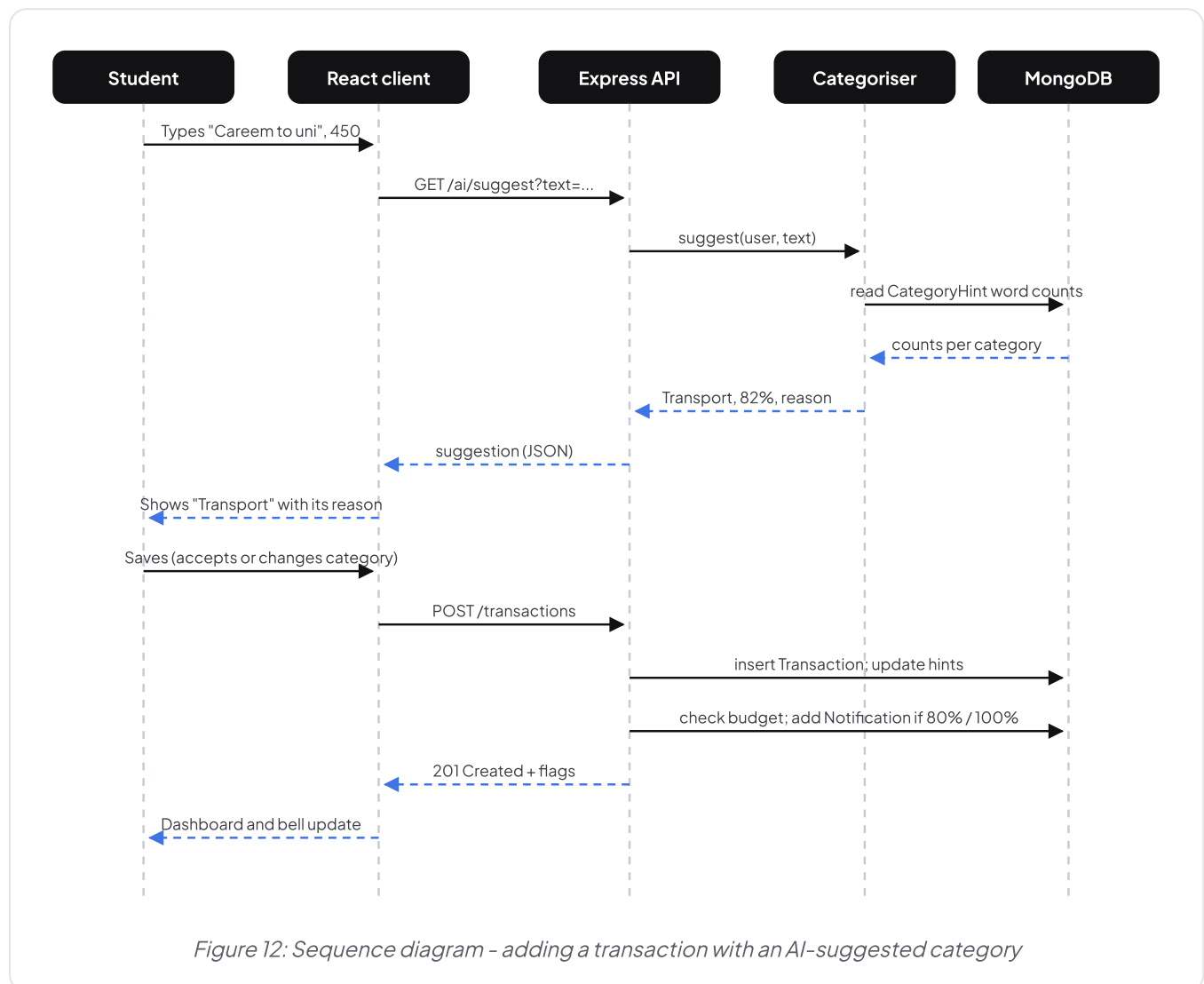


Figure 11: Data flow diagram, level 1

At level 1, the system splits into six processes that share seven data stores. Recording a transaction (2.0) asks the categoriser (3.0) for a suggestion, which learns from the saved result (D4). Budgets (4.0) read spending from the transactions store and write alerts to the notifications store. Reports, insights and tips (5.0) read transactions and categories and store insight and tip history (D6). Administration (6.0) maintains default categories and pushes announcements to every student's notifications.

4.6 Sequence diagram

The sequence below follows one of the most frequent actions - adding an expense - from the student's typing to the saved record, showing where the categoriser and the budget check take part.

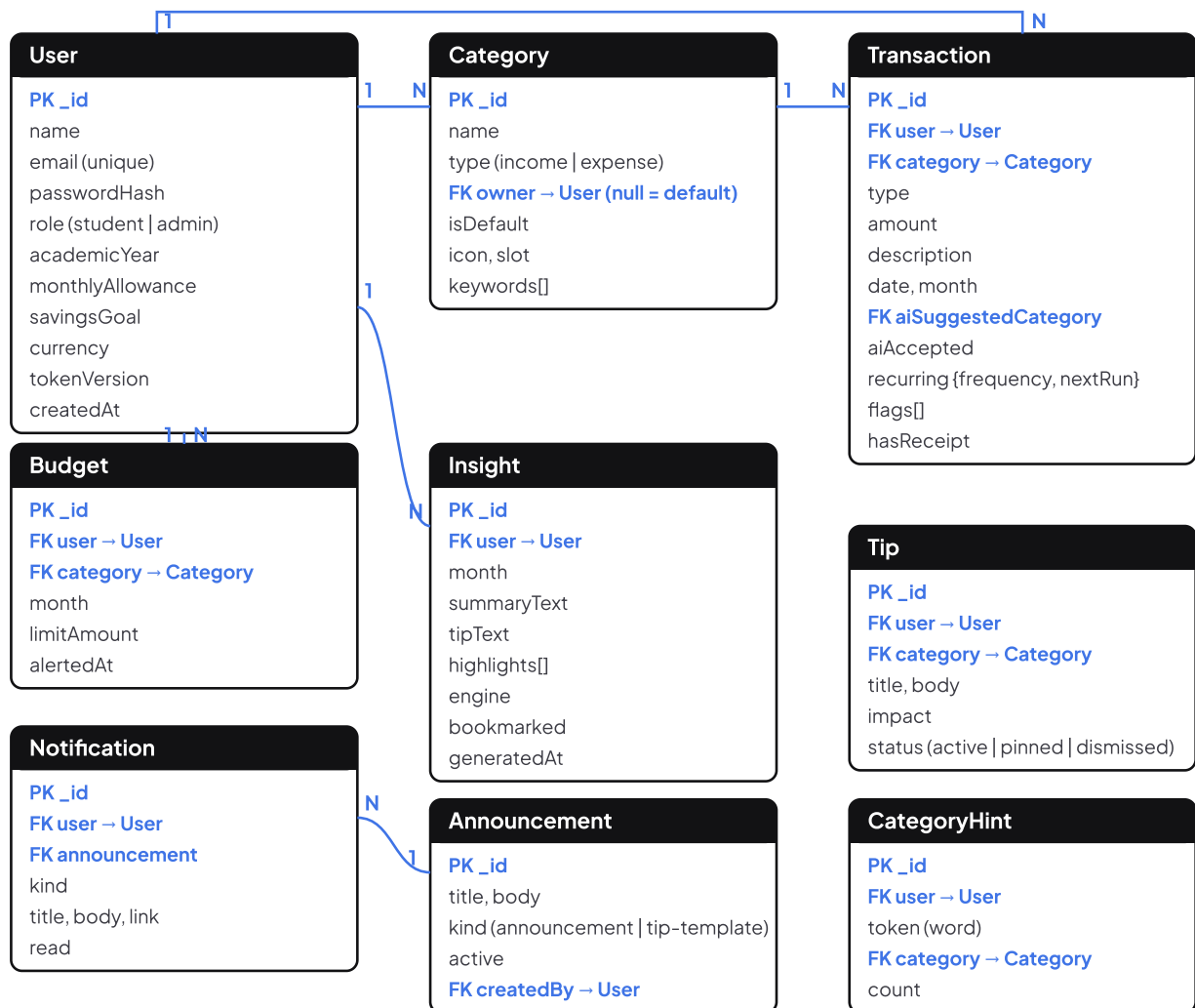


5

Database Design

Collections, fields and relationships in MongoDB.

Campus Coin uses MongoDB, a document database. Each entity below is a collection; relationships are stored as references (ObjectId) to the related document, which is the MongoDB equivalent of a foreign key. Every collection also records `createdAt` and `updatedAt` automatically.



Every collection except Category defaults and Announcement belongs to one User (1 : N); see the relationship table below.

Figure 13: Entity-relationship diagram (MongoDB collections)

5.1 Relationships

Relationship	Type	Meaning
User → Transaction	1 : N	A student owns many transactions

Relationship	Type	Meaning
Category → Transaction	1 : N	Each transaction belongs to one category
User → Category	1 : N	A student's own categories (owner); defaults have no owner
User → Budget, Category → Budget	1 : N	One cap per category per month
User → Insight	1 : N	One insight per month, kept as history
User → Tip, Notification, CategoryHint	1 : N	Personal tips, alerts and learned word pairings
Announcement → Notification	1 : N	An announcement is copied into each student's bell

5.2 Collections

User

Field	Type	Key	Description
<code>_id</code>	ObjectId	PK	Unique identifier
<code>name</code>	String		Full name
<code>email</code>	String	Unique	Login email
<code>passwordHash</code>	String		bcrypt hash of the password (never returned to the client)
<code>role</code>	String		student or admin
<code>academicYear</code>	String		Study level: Matric, O/A Levels, Intermediate, university year, Masters, MPhil, PhD
<code>institution</code>	String		School, college or university
<code>monthlyAllowance</code>	Number		Monthly allowance baseline
<code>savingsGoal</code>	Number		Monthly savings goal
<code>currency</code>	String		PKR, USD, EUR, GBP, INR or AED
<code>avatar, avatarColor</code>	String		Profile photo (small data URL) or initial colour
<code>preferences</code>	Object		theme, fontScale, reducedMotion, alertsEnabled
<code>disabled</code>	Boolean		Set by the administrator to block sign-in
<code>tokenVersion</code>	Number		Session version; raised to end all sessions
<code>failedLogins, lockUntil</code>	Number, Date		Sign-in lockout
<code>resetTokenHash, resetTokenExpires</code>	String, Date		One-time password link (hash only)

Field	Type	Key	Description
lastLoginAt, passwordChangedAt	Date		Account history

Category

Field	Type	Key	Description
_id	ObjectId	PK	Unique identifier
name	String		e.g. Food, Transport, Hostel/Rent
type	String		income or expense
owner	ObjectId	FK → User	Empty for a system default
isDefault	Boolean		System default or student-created
icon, slot	String, Number		Picture and colour
keywords	String[]		Seed words for the categoriser
archived	Boolean		Hidden but kept for old transactions

Transaction

Field	Type	Key	Description
_id	ObjectId	PK	Unique identifier
user	ObjectId	FK → User	Owner
category	ObjectId	FK → Category	Chosen category
type	String		income or expense
amount	Number		Amount (positive)
description, note	String		Free text entered by the student
date, month	Date		Date of the transaction and its month (for fast monthly queries)
aiSuggestedCategory	ObjectId	FK → Category	Category the assistant suggested
aiAccepted	Boolean		Whether the suggestion was kept (accuracy tracking)
recurring	Object		enabled, frequency (weekly/monthly), nextRun
flags	String[]		large, duplicate
source	String		manual, csv, recurring or receipt
receipt, hasReceipt	String, Boolean		Compressed receipt photo

Budget

Field	Type	Key	Description
<code>_id</code>	ObjectId	PK	Unique identifier
<code>user</code>	ObjectId	FK → User	Owner
<code>category</code>	ObjectId	FK → Category	Capped category
<code>month</code>	Date		Month the budget applies to
<code>limitAmount</code>	Number		The cap
<code>alertedAt</code>	Number		Last alert level sent (0, 80 or 100)

Insight

Field	Type	Key	Description
<code>_id</code>	ObjectId	PK	Unique identifier
<code>user</code>	ObjectId	FK → User	Owner
<code>month</code>	Date		Month covered
<code>summaryText</code>	String		Narrative summary
<code>tipText</code>	String		Actionable advice
<code>highlights</code>	Array		Categories that moved, with change %
<code>stats</code>	Object		income, expense, balance, savings rate, top category
<code>engine</code>	String		statistical or llm
<code>bookmarked</code>	Boolean		Saved by the student
<code>generatedAt</code>	Date		Time of generation

Tip, Notification, Announcement and CategoryHint

Collection	Main fields	Purpose
Tip	user, key, title, body, impact, category, status (active/pinned/dismissed)	Saving tips ranked by impact
Notification	user, kind, title, body, link, read, announcement	Budget alerts, anomaly checks, news in the bell
Announcement	title, body, kind (announcement/tip-template), active, createdBy	System-wide notices and tip templates
CategoryHint	user, token (word), category, count	What the categoriser has learned for each student

6

Functional Requirements and How They Are Met

Every SRS requirement next to its implementation.

SRS requirement	Implementation in Campus Coin
Student registration and login; separate admin login	/register and /login for students; /admin/login refuses any non-admin account
Secure session management	Signed JWT with session version, logout, "sign out everywhere", immediate block for disabled accounts
Password recovery by tokenised link	One-time 1-hour link emailed through Gmail SMTP; password change also works only by link
Editable profile: name, academic year, allowance, savings goal	Account settings, plus institution, currency, photo and preferences
Bulk import from CSV	Import wizard with preview and batch category suggestions; CSV export too
Default and personal income/expense categories	All 12 defaults seeded; "Manage own categories" to add, edit, delete
Dashboard: greeting, balance, quick add	Time-of-day greeting card, money in/out/kept, quick-add tiles for chai, rickshaw, printing, allowance
Saving tips and highlights on the dashboard	"Worth doing" and "Coin's insight" cards
Widgets: This Month's Top Category, Budget vs. Actual	Both present as their own cards
Quick-add, recurring entries, edit and delete	Transaction form, weekly/monthly repeats written when due, edit/delete with history kept
AI category suggestion that learns and can be overridden	Per-student word-to-category model; corrections are learned; accuracy tracked
Batch categorisation on CSV import	Every imported row gets a suggested category in the preview
Category-wise, six-month, daily and weekly reports; filters	Reports page with donut, month bars, daily and weekly views, filters by date, category, source
Export report as PDF or image	Report export to PDF and PNG
AI monthly insight, flagged growth, advice, history	Insights page with summary, month score, category changes, habits, stored history

SRS requirement	Implementation in Campus Coin
Saving tips ranked by impact; pin or dismiss	Tips page; every tip states its monthly value
Monthly budget per category, live progress, alerts	Budgets page; notifications at 80% and 100%
Bookmark tips and insights; share by email	Pin tips, bookmark insights, email the monthly summary
Admin: default categories, announcements, users, statistics	Admin control panel with all four areas
Recently viewed, forecast, large/duplicate detection	Recent list, least-squares forecast, anomaly flags
Dark mode, font size, breadcrumbs, loading indicators	Theme toggle, four text sizes, breadcrumbs, loader and shimmering placeholders
AI chatbot	Built-in assistant "Coin", answering from the student's own data
OCR for receipts (optional AI/ML)	Receipt photo read in the browser: amount, shop and date filled in

7

Non-Functional Requirements

Quality attributes and how they are achieved.

Requirement	How it is achieved
Safe to use	No downloads except exports the student asks for; uploaded pictures are checked for type and size on the server
Accessibility	Readable font, strong contrast, keyboard focus, labels for screen readers, dark mode, text-size control, reduced-motion support
User-friendliness	A four-step getting-started checklist for new accounts, quick-add buttons, category suggested automatically
Operability and reliability	65 automated end-to-end checks run against the local and the live site
Performance	The dashboard loads in one request; charts are lightweight SVG; indexes on user and month
Scalability	Stateless API on Vercel serverless functions and MongoDB Atlas, both scaling with traffic
Security	Authentication on every data route; each student can read only their own data; hashed passwords; HTTPS
Availability	Hosted on Vercel and MongoDB Atlas, available 24/7
Compatibility	Tested in Chrome and on phone, tablet and laptop widths; works in current browsers

8 User Interface

The main screens of the application.

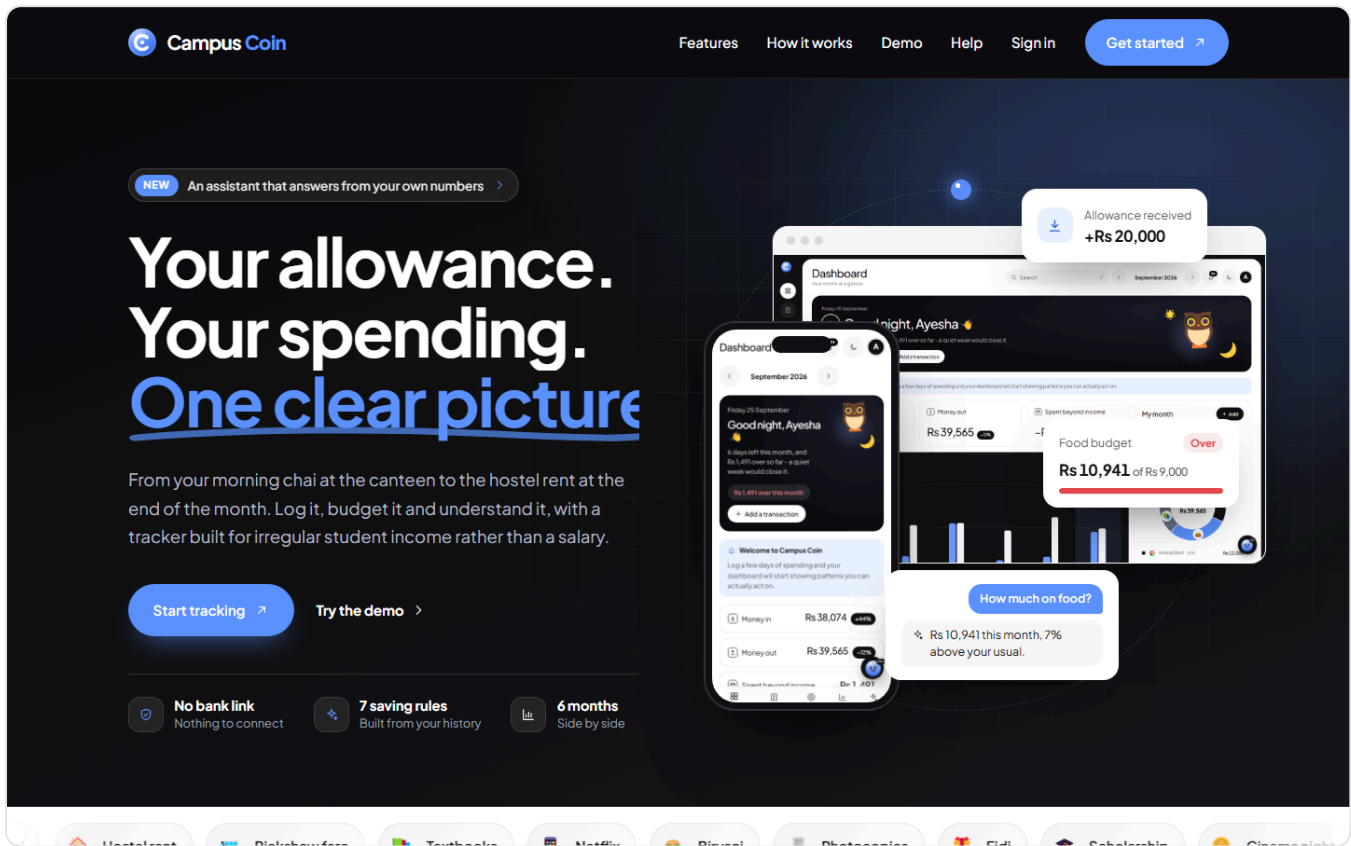


Figure 14: Landing page

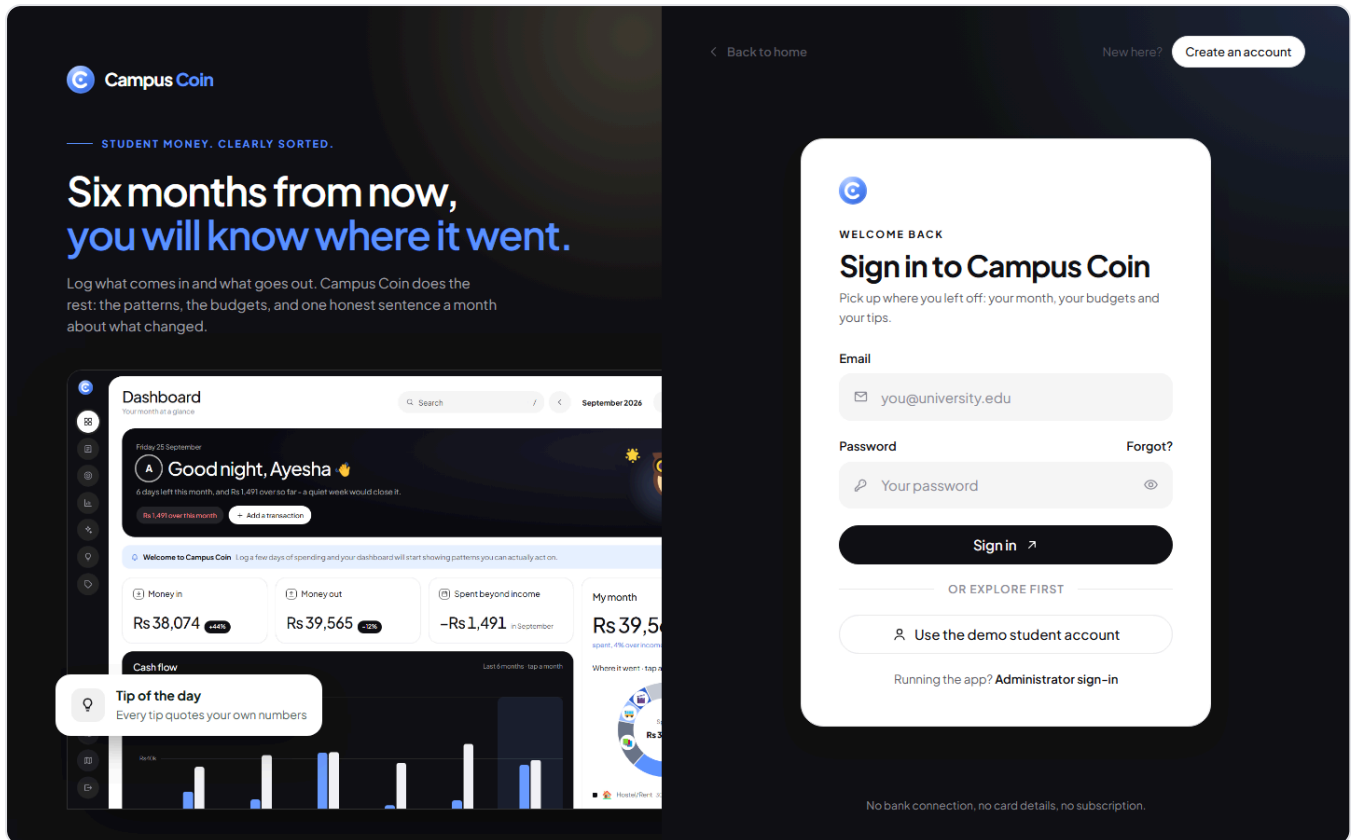


Figure 15: Student sign-in with the demo account button

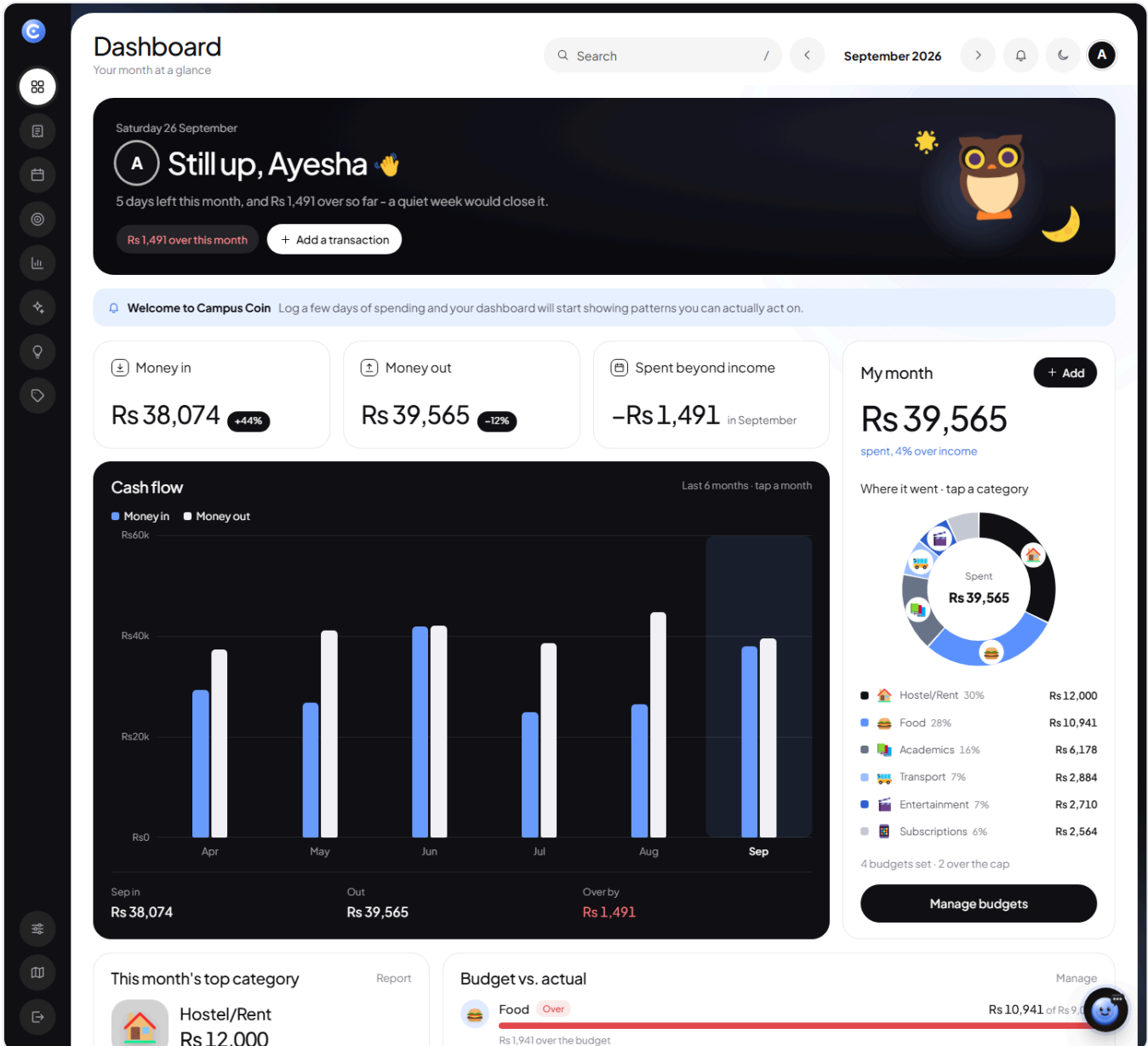


Figure 16: Dashboard: greeting, money in and out, cash flow, where it went, top category, budget vs actual

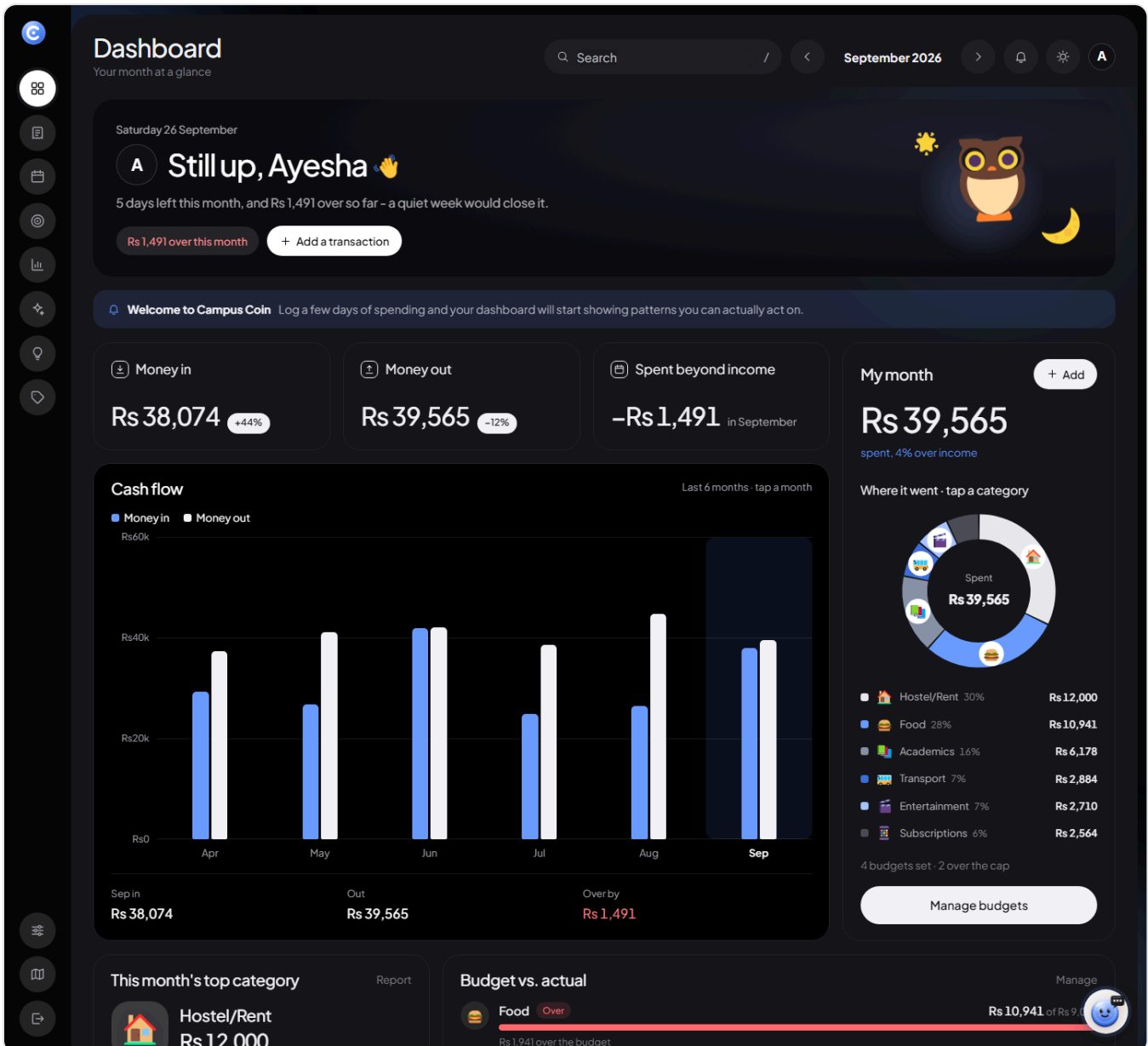


Figure 17: Dashboard in dark mode

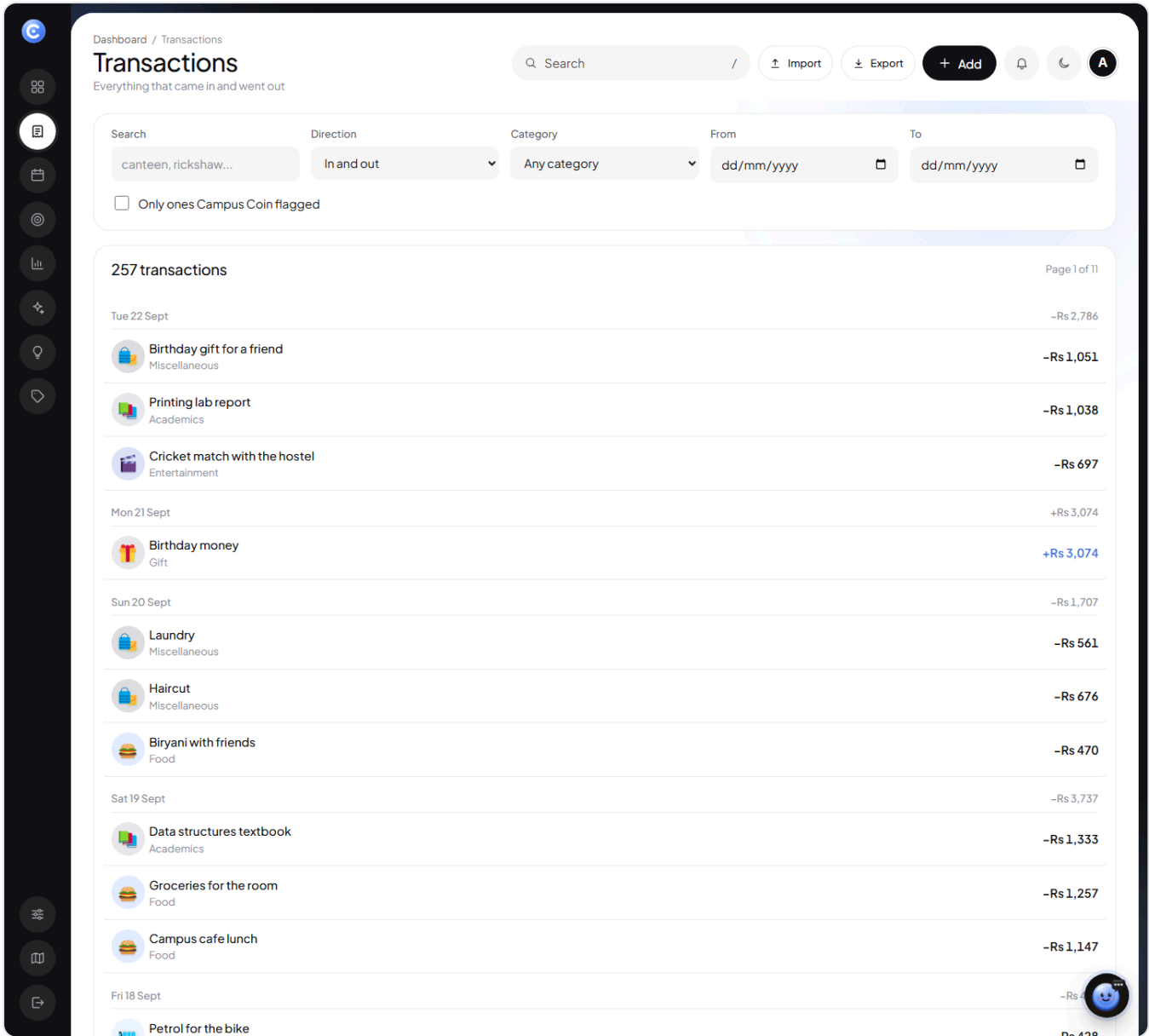


Figure 18: Transactions with category pictures and merchant logos

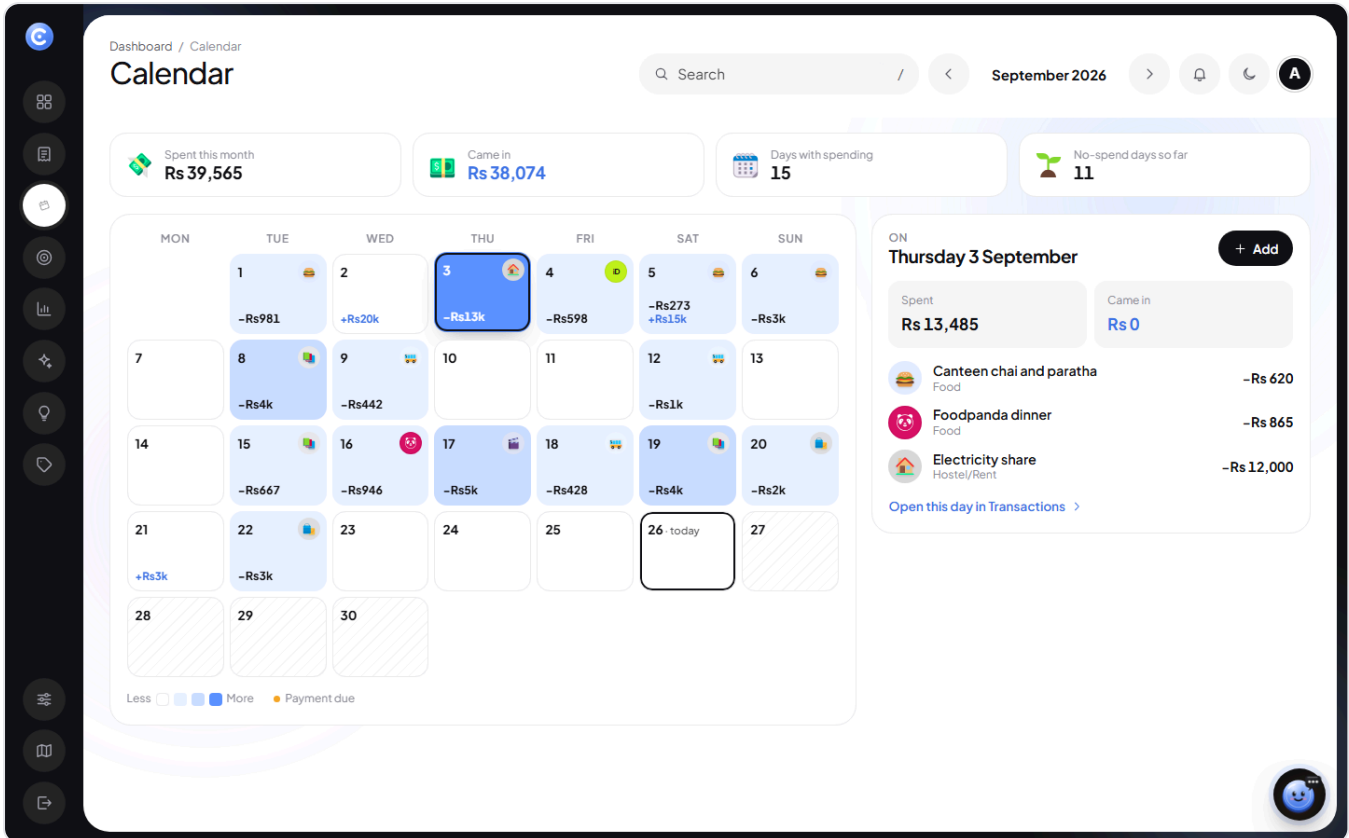


Figure 19: Money calendar: each day's spending and the selected day's transactions

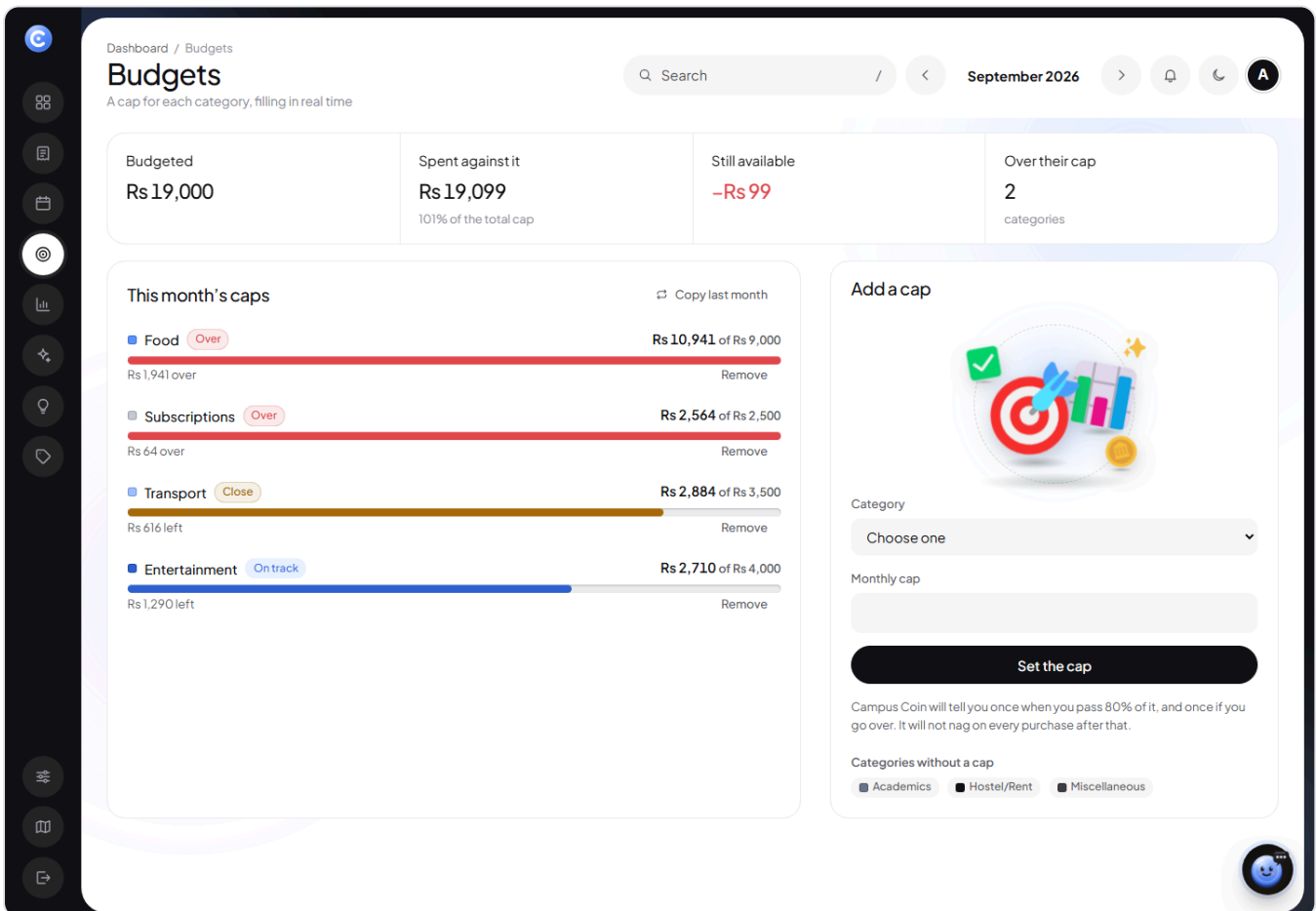


Figure 20: Budgets with live progress bars

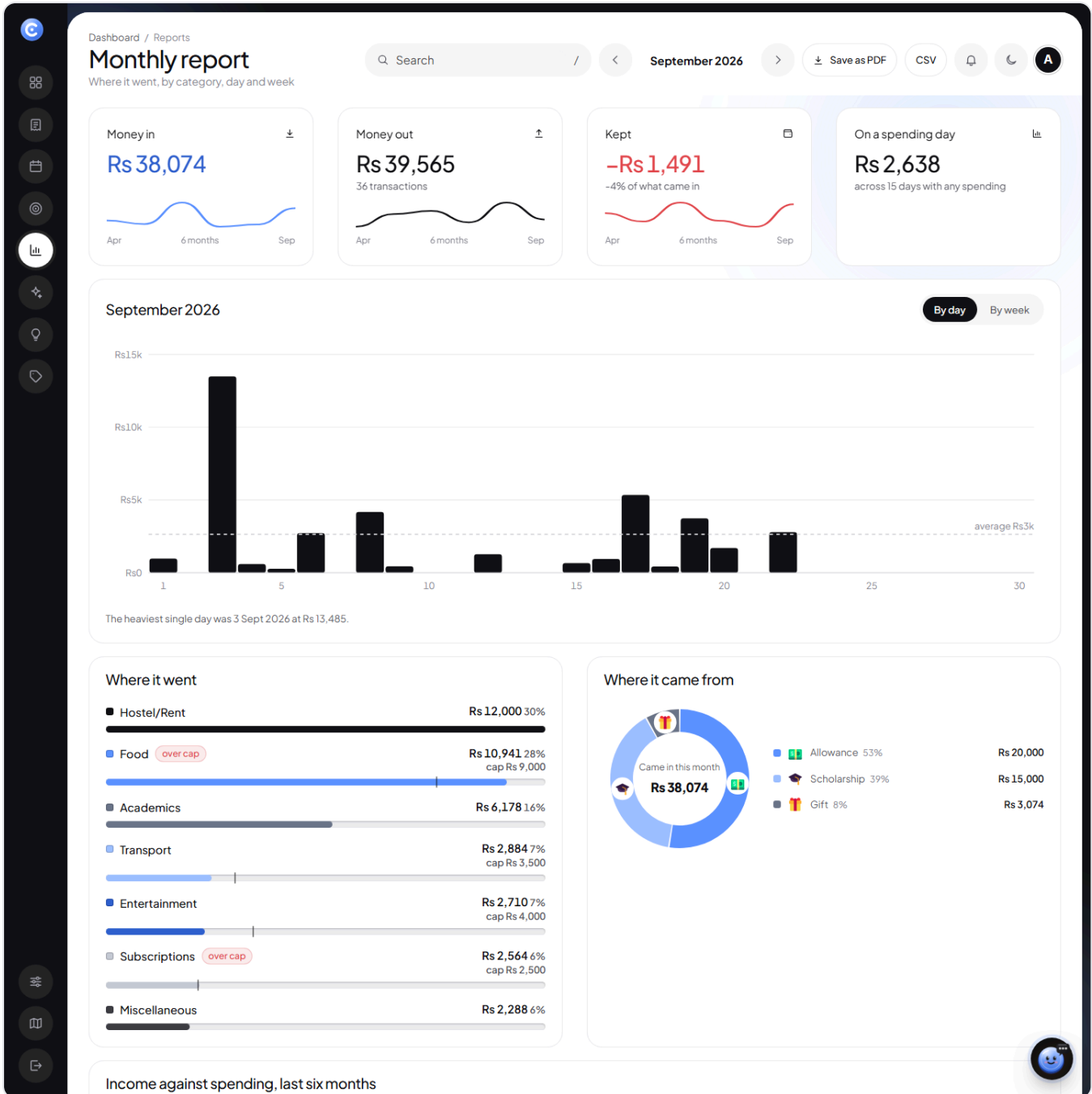


Figure 21: Monthly report

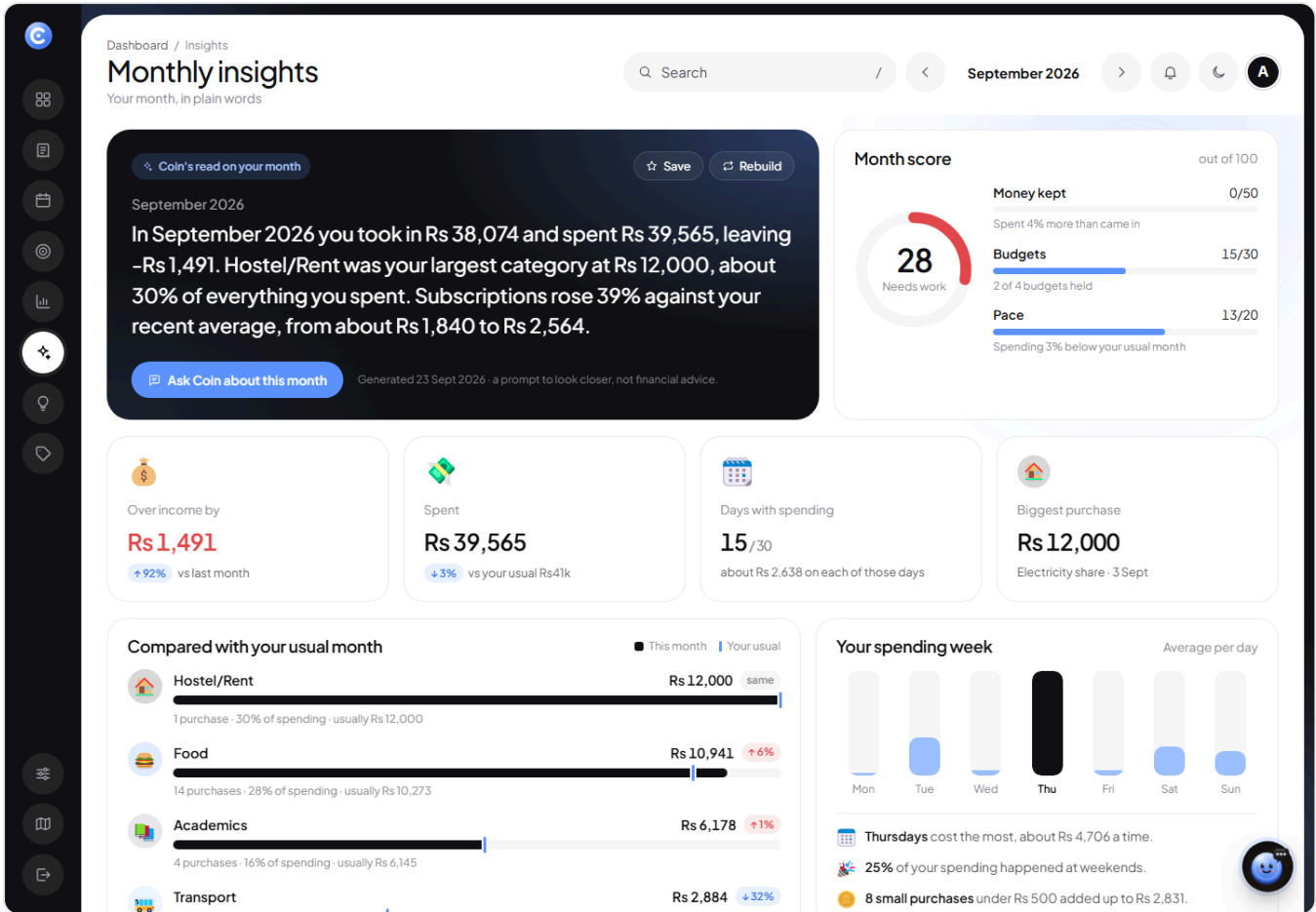


Figure 22: Monthly insights

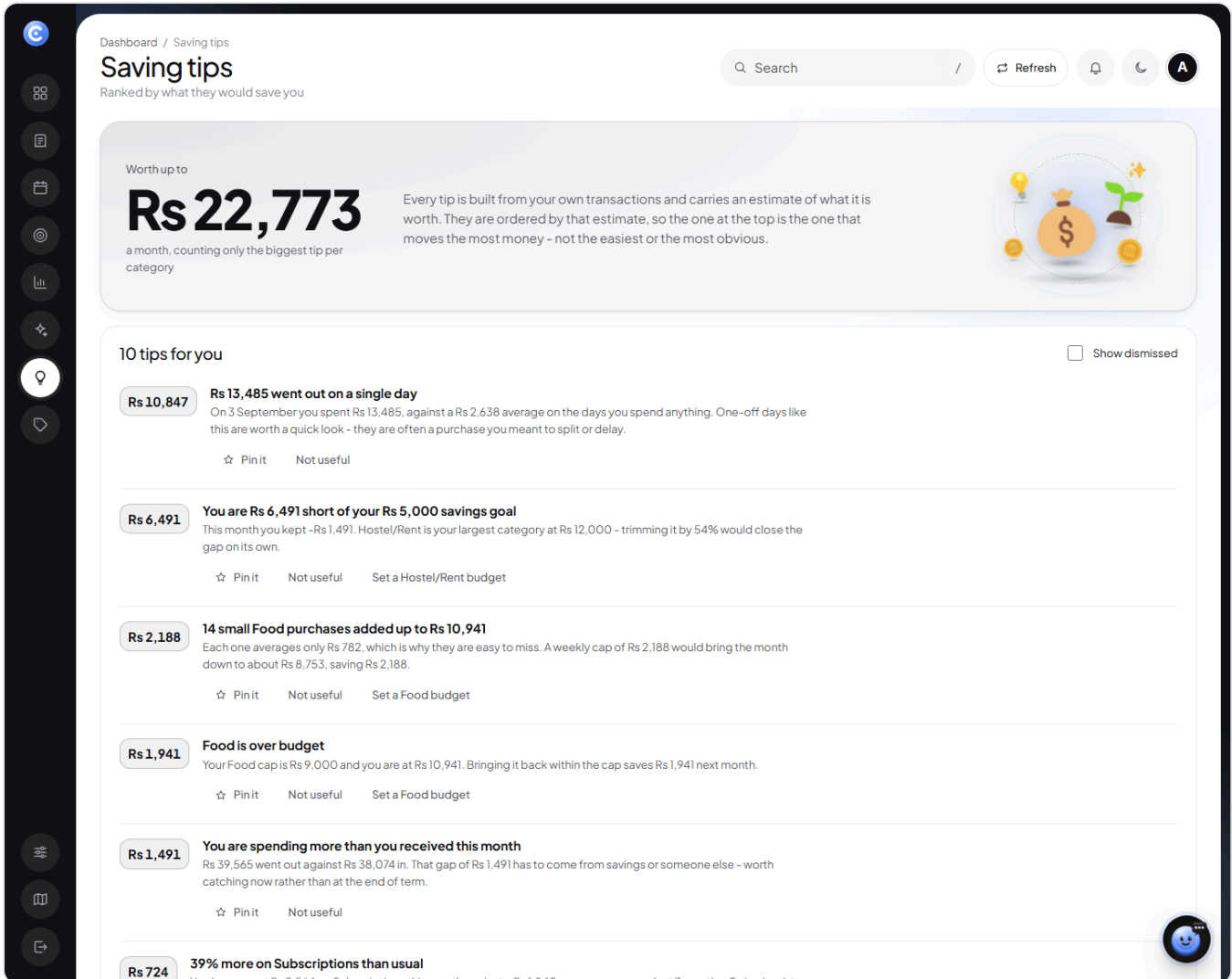


Figure 23: Saving tips ranked by impact

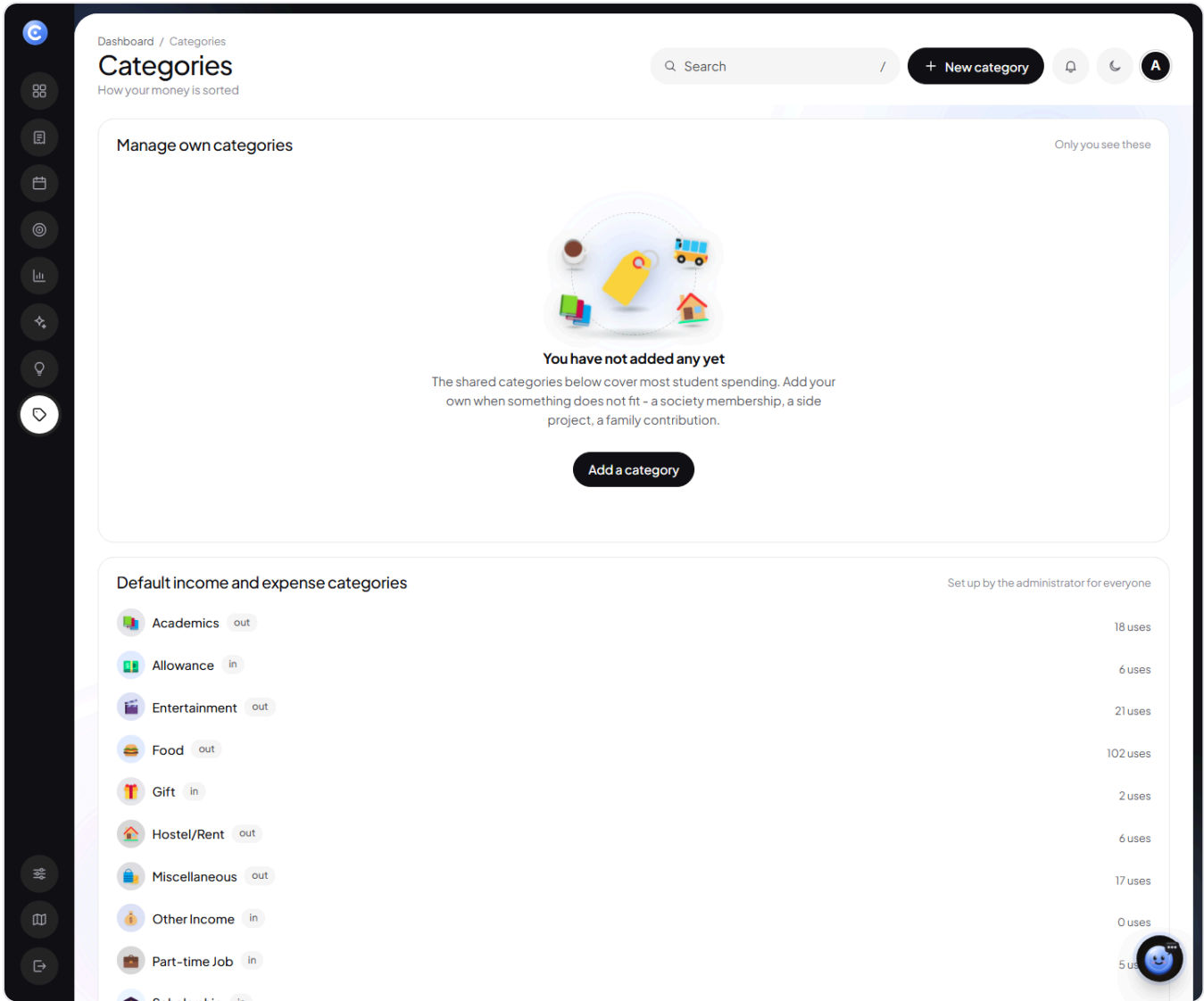


Figure 24: Categories: manage own categories and the defaults

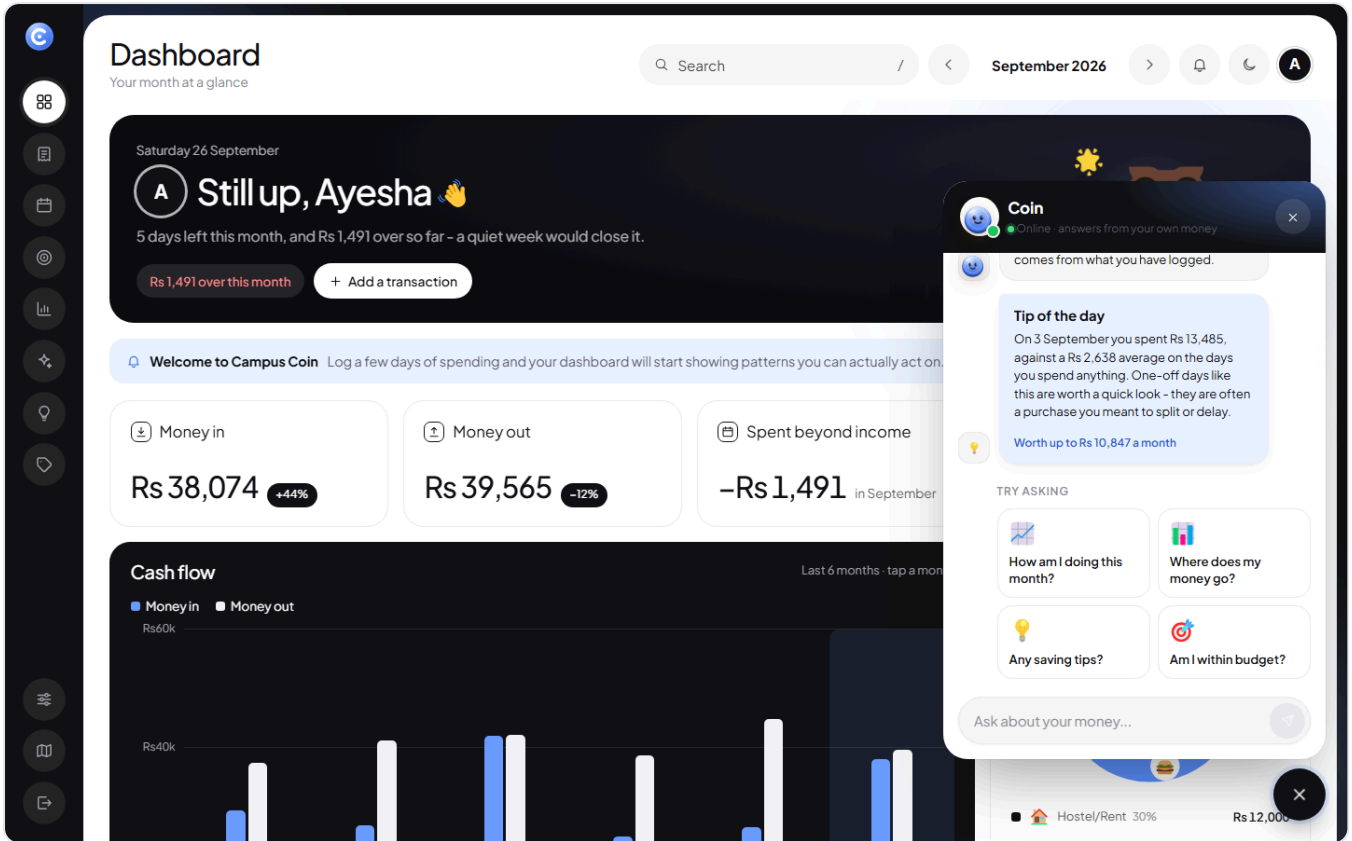


Figure 25: Coin, the chat assistant

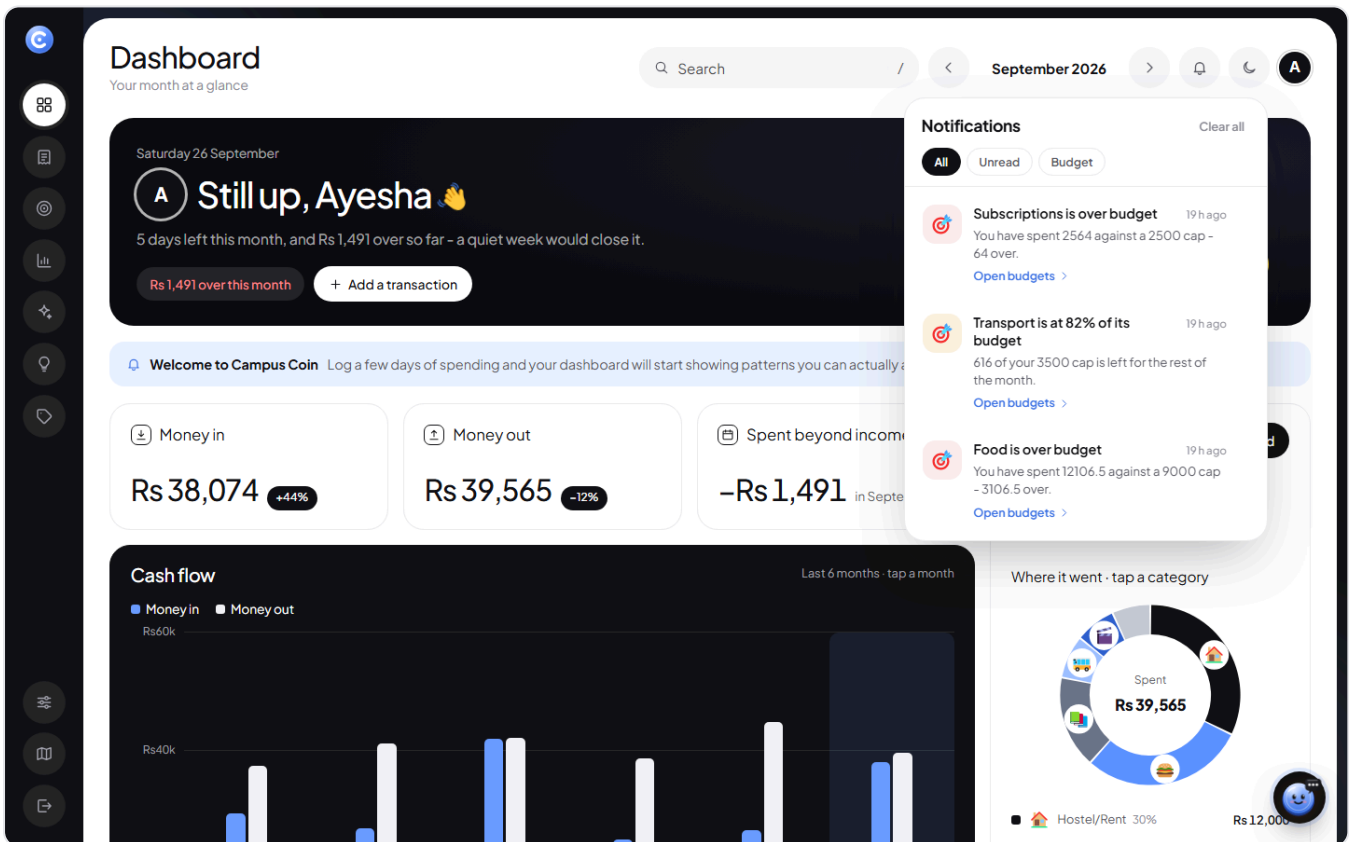


Figure 26: Notifications panel

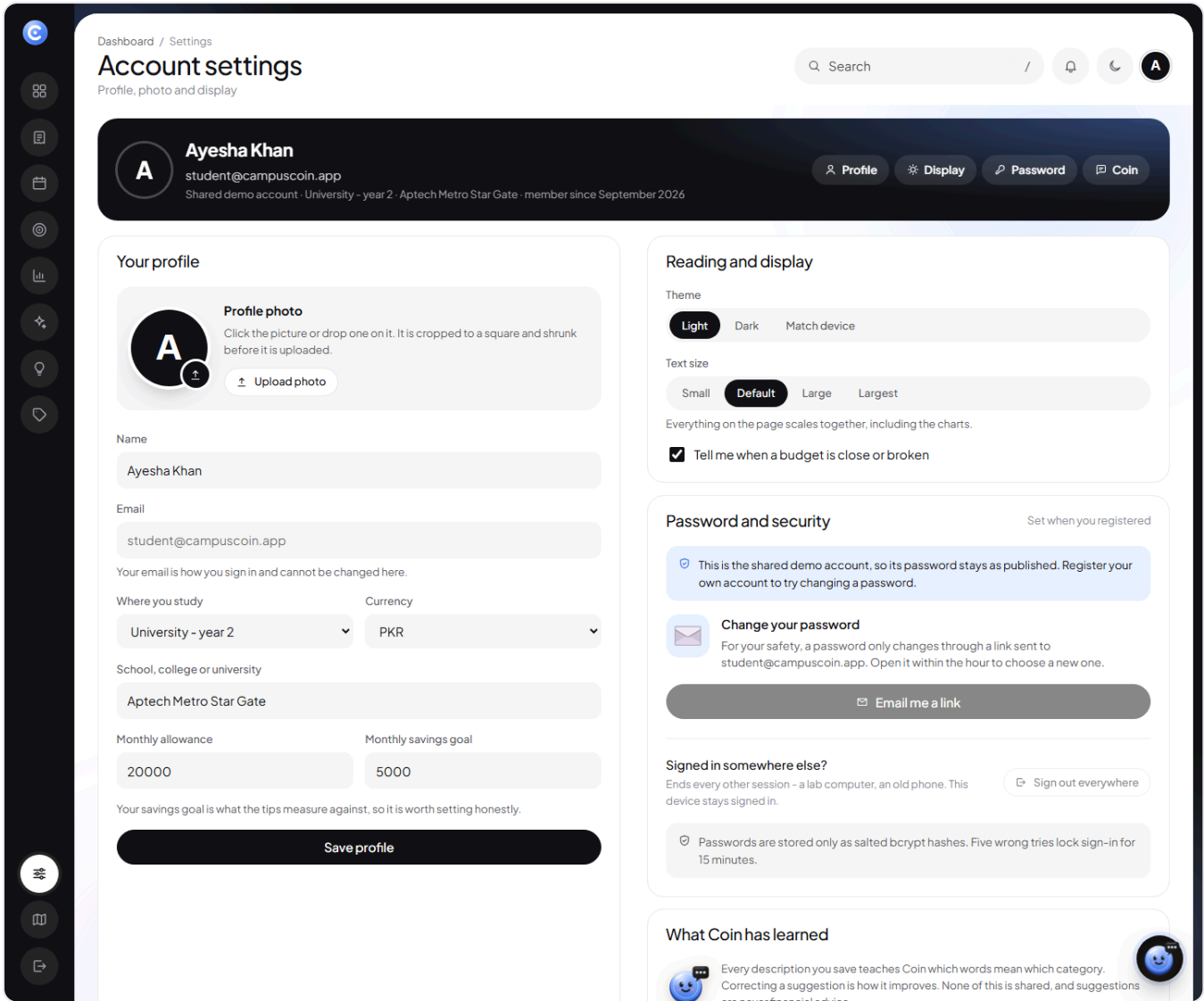


Figure 27: Account settings

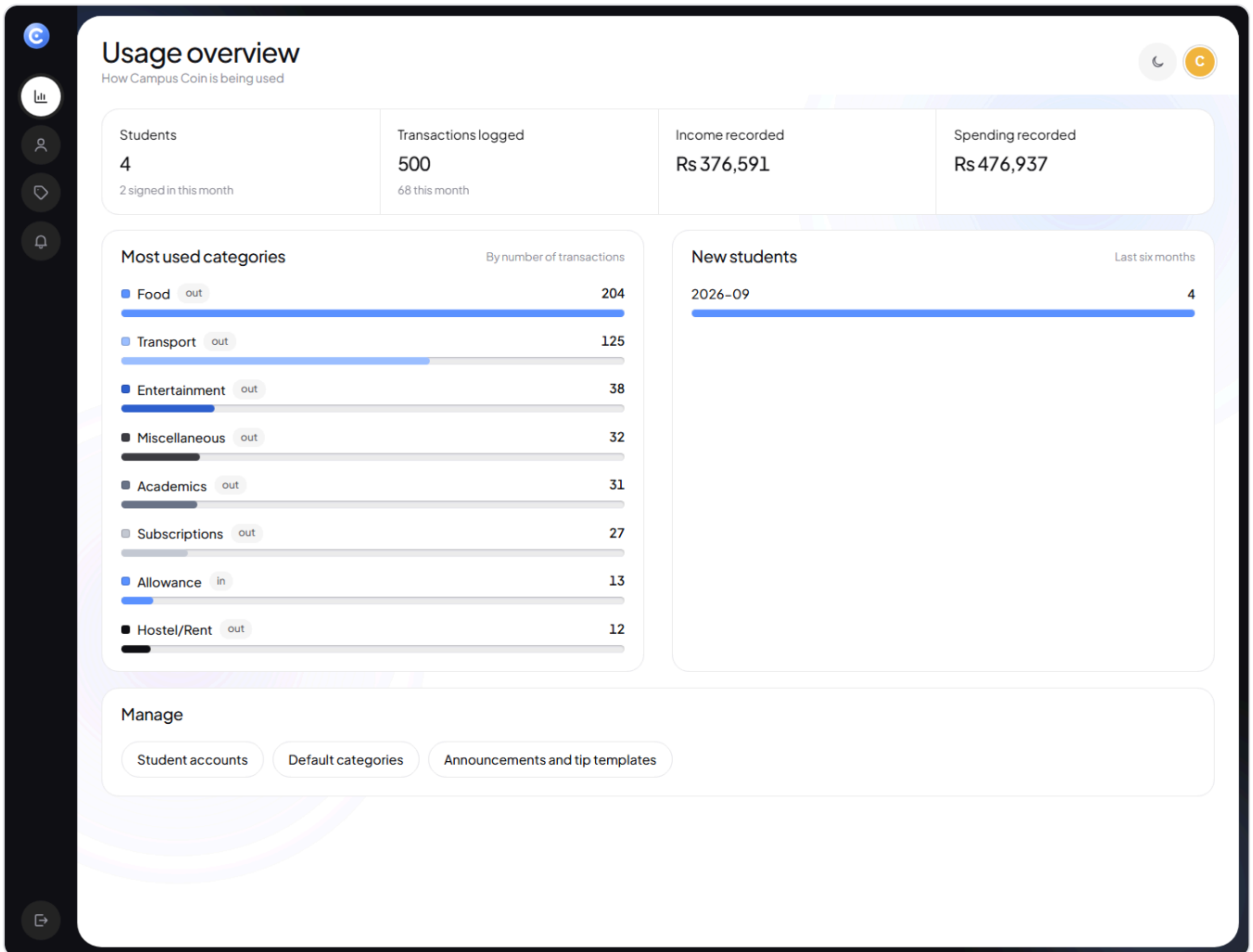


Figure 28: Admin control panel: usage statistics

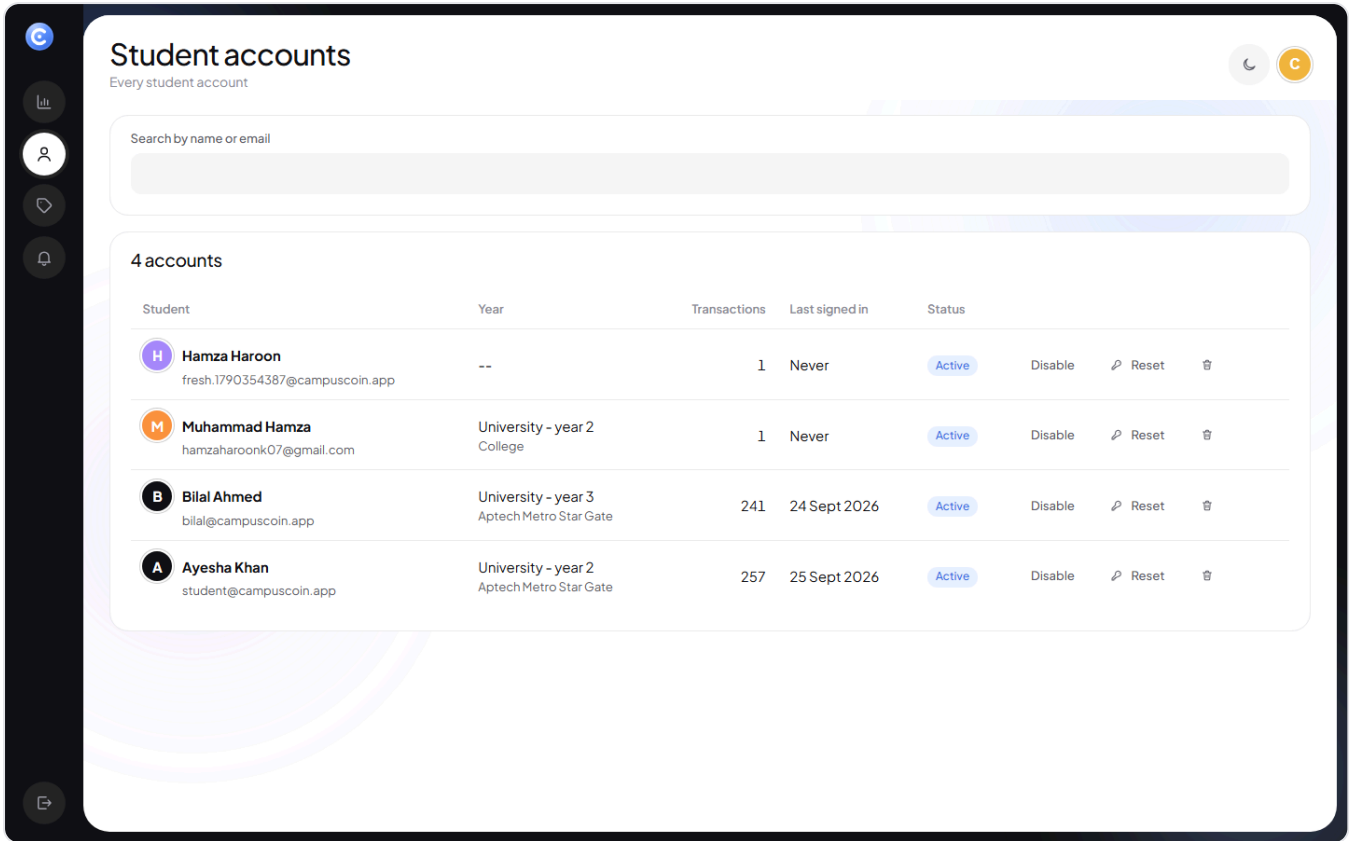


Figure 29: Admin: student accounts

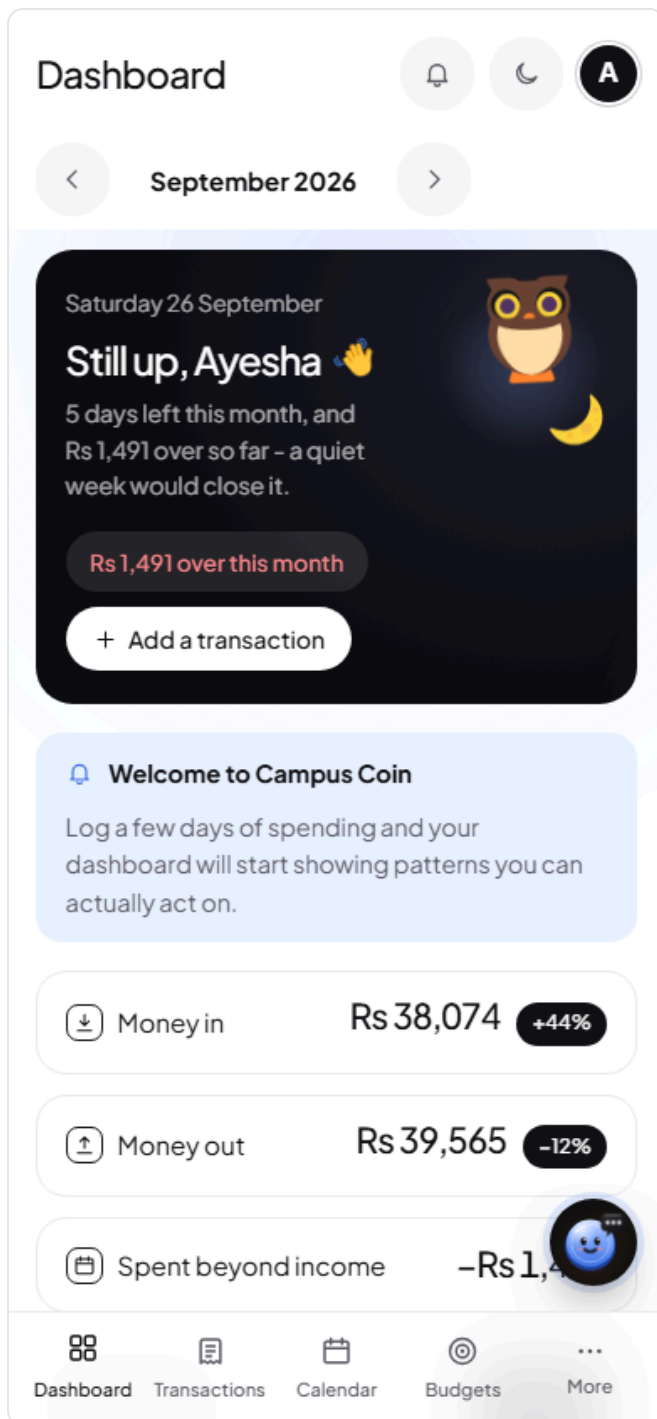


Figure 30: Dashboard on a phone

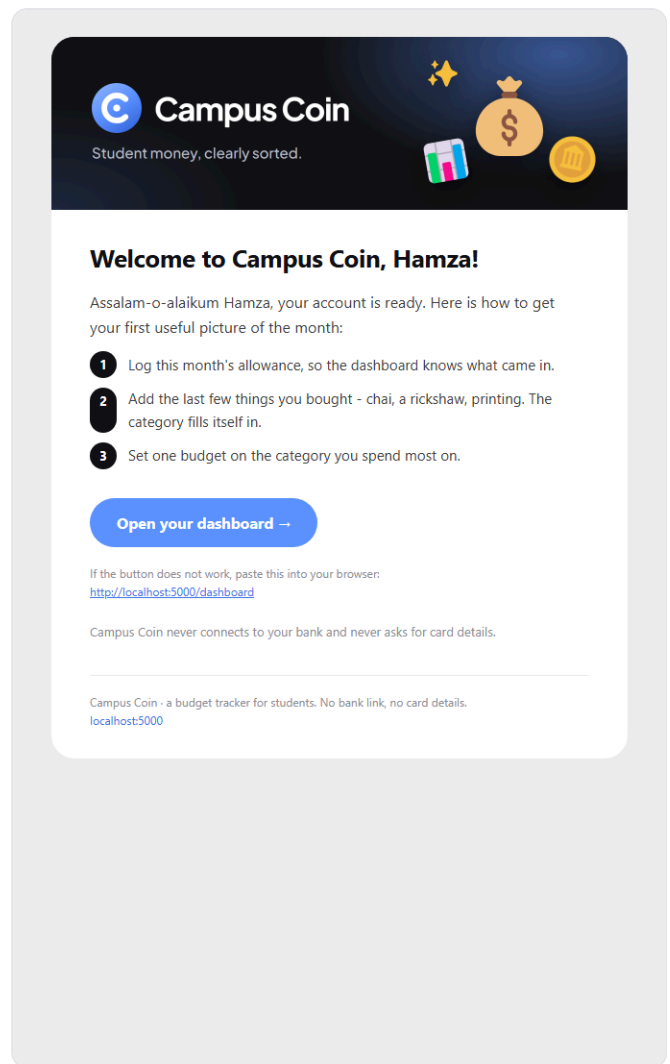


Figure 31: Welcome email

The data used for testing, and the test cases with results.

9.1 Test data

The first start of the application seeds the database automatically, so every screen can be tested with realistic data straight away:

- **12 default categories:** Allowance, Part-time Job, Scholarship, Gift, Other Income; Food, Transport, Hostel/Rent, Academics, Subscriptions, Entertainment, Miscellaneous – each with keywords for the categoriser.
- **Two demo students with six months of history** (about 250 transactions each), generated with a fixed random seed so the data is the same on every install.
- **Demo budgets** for the current month: Food Rs 9,000, Subscriptions Rs 2,500, Transport Rs 3,500 and Entertainment Rs 4,000 – two of them deliberately exceeded so alerts can be seen.
- **Recurring entries** such as the monthly allowance and Spotify Premium, and a welcome announcement.
- **Two sample receipts** (a café bill and a bookshop bill) in `client/public/samples` for testing the receipt scanner.

Category	Seeded pattern per month	Example descriptions
Allowance	Rs 20,000 on the 2nd	Monthly allowance from home
Part-time Job	Rs 3,000 – 9,000, most months	Tutoring payment, Weekend shift, Freelance design work
Scholarship	Rs 15,000 every third month	Merit scholarship instalment
Gift	Rs 1,000 – 5,000, occasionally	Birthday money, Eidi from uncle
Food	14 – 22 purchases, Rs 180 – 900	Canteen chai and paratha, Foodpanda dinner, Biryani with friends
Transport	8 – 16 trips, Rs 120 – 600	Rickshaw to campus, Careem to the station, InDrive home
Hostel/Rent	Rs 12,000 once	Monthly hostel rent, PTCL wifi bill
Academics	1 – 4 purchases, Rs 400 – 3,500	Printing lab report, Data structures textbook
Subscriptions	2 – 3, Rs 500 – 1,500	Spotify Premium, Netflix share, ChatGPT Plus share
Entertainment	2 – 5, Rs 600 – 2,500	Cinema ticket, Cricket match with the hostel
Miscellaneous	2 – 5, Rs 200 – 1,800	Haircut, Laundry, Medicine from the pharmacy

Sample CSV import file

```
date,description,amount,type
2026-09-01,Monthly allowance,20000,income
2026-09-03,Chai at the canteen,80,expense
2026-09-04,Rickshaw to campus,150,expense
2026-09-05,Printing notes,120,expense
2026-09-06,Foodpanda dinner,650,expense
```

9.2 Test cases

An automated end-to-end test (`server/test/e2e.mjs`) registers a fresh student, works through every functional requirement against the running API, then deletes the account. It is run against the local server and the live site; the latest run passed all 65 checks on both.

#	Area	Test case	Result
1	Auth	Register a student with profile fields	Pass
2	Auth	Duplicate email is refused	Pass
3	Auth	Log in	Pass
4	Auth	Wrong password is refused	Pass
5	Auth	Admin sign-in refuses a student	Pass
6	Auth	Protected routes need a session	Pass
7	Auth	Edit profile (name, year, allowance, goal)	Pass
8	Auth	Year of study covers school, college and university	Pass
9	Auth	Theme and font size saved to the account	Pass
10	Auth	Weak passwords are refused	Pass
11	Auth	Password cannot change without an emailed link	Pass
12	Auth	Change password by link ends other sessions	Pass
13	Auth	Sign out everywhere	Pass
14	Auth	The shared demo password cannot be changed	Pass
15	Auth	Password recovery by tokenised link	Pass
16	Categories	Default income and expense categories exist	Pass
17	Categories	Create a personal category	Pass
18	Categories	Edit a personal category	Pass
19	Categories	Delete a personal category	Pass
20	Categories	Default categories cannot be edited by a student	Pass
21	Transactions	Quick-add income	Pass
22	Transactions	Quick-add expenses	Pass

#	Area	Test case	Result
23	Transactions	Invalid amount is refused	Pass
24	Transactions	Recurring entry (monthly subscription)	Pass
25	Transactions	Edit a transaction	Pass
26	Transactions	Delete a transaction	Pass
27	Transactions	List, search and filter	Pass
28	Transactions	Recently viewed/edited list	Pass
29	Anomalies	Unusually large expense is flagged	Pass
30	Anomalies	Duplicate transaction is flagged	Pass
31	AI assistant	Suggests a category as you type	Pass
32	AI assistant	Learns from a correction	Pass
33	AI assistant	Override is recorded (accuracy tracked)	Pass
34	Import	CSV preview with batch category suggestions	Pass
35	Import	CSV commit	Pass
36	Import	CSV export	Pass
37	Budgets	Set a monthly budget per category	Pass
38	Budgets	Real-time progress (spent vs cap)	Pass
39	Budgets	In-app notification when a category nears/exceeds budget	Pass
40	Budgets	Copy budgets to next month	Pass
41	Dashboard	Greeting data, balance, top category, budget vs actual, tips	Pass
42	Reports	Category-wise monthly report	Pass
43	Reports	Calendar groups the month by day	Pass
44	Reports	Daily and weekly summaries	Pass
45	Reports	Income vs expense, last six months	Pass
46	Reports	Filter by date range, category and income source	Pass
47	Reports	Forecast needs 3 months (honest refusal for a new account)	Pass
48	Reports	Flagged transactions list	Pass
49	Insights	Generate the monthly narrative insight	Pass
50	Insights	Insight history is stored	Pass
51	Insights	Bookmark an insight	Pass
52	Insights	Share the summary by email	Pass
53	Tips	Tips generated from own history, ranked by impact	Pass
54	Tips	Pin and dismiss a tip	Pass
55	Chat	Answers questions from own data	Pass
56	Admin	Separate admin login	Pass

#	Area	Test case	Result
57	Admin	Students cannot reach admin routes	Pass
58	Admin	Usage statistics (active users, transactions, top categories)	Pass
59	Admin	View and search users	Pass
60	Admin	Disable a user blocks login, enable restores it	Pass
61	Admin	Reset a user password	Pass
62	Admin	Repeated wrong passwords lock the account	Pass
63	Admin	Add, edit and remove a default category	Pass
64	Admin	Announcements and tip templates	Pass
65	Admin	Remove a user account (clean-up)	Pass

9.3 Manual testing

Scenario	Steps	Expected result
Quick add	Dashboard → Chai tile → enter 80 → Save	Saved under Food; dashboard totals update
AI suggestion	Add transaction → type "Careem to uni"	Transport suggested with its reason; can be changed
Receipt scan	Add transaction → Scan receipt → sample café bill	Amount, shop and date filled in
Budget alert	Set Food budget below current spending	Notification "Food is over budget" appears in the bell
Password link	Settings → Email me a link → open email → set password	Signed in with the new password; other devices signed out
Lockout	Enter a wrong password five times	Sign-in refused for 15 minutes
Phone layout	Open on a 390px-wide screen	Bottom tab bar, stacked cards, no sideways scrolling

10 Project Installation Instructions

How to install and run Campus Coin on a new machine.

10.1 Requirements

- Node.js 18 or newer (includes npm) - <https://nodejs.org>
- A MongoDB server is optional: with none configured, the application starts its own embedded MongoDB in `server/data/db`.
- Any current browser (Chrome, Edge, Firefox, Safari).

10.2 Steps

```
# 1. Get the code (or unzip the submitted folder)
git clone https://github.com/hamzaharoonk07-cloud/campuscoin.git
cd campuscoin

# 2. Install the server and client packages
npm run setup

# 3. Create the settings file (every value in it is optional)
copy server\.env.example server\.env          (Windows)
cp server/.env.example server/.env             (macOS / Linux)

# 4. Build the React front end
npm run build

# 5. Start the application
npm start
```

Open **<http://localhost:5000>** in a browser. On the first start the database is seeded with the default categories, the administrator and the two demo students, so every screen has data immediately.

10.3 Development mode

```
npm run dev          # API on http://localhost:5000
npm run dev:client    # React on http://localhost:5173 (hot reload)
```

10.4 Optional settings (server/.env)

Variable	Purpose
MONGO_URI	Use an external MongoDB (local or Atlas) instead of the embedded one
JWT_SECRET	Secret used to sign session tokens
SMTP_HOST , SMTP_PORT , SMTP_USER , SMTP_PASS , MAIL_FROM	Send real emails (for Gmail: smtp.gmail.com, 465, the address and an app password). Without them, reset links are shown on screen on a local machine.
ANTHROPIC_API_KEY	Let Claude reword monthly summaries; without it the built-in writer is used
CRON_SECRET	Protect the daily recurring-transactions job on Vercel

10.5 Running the tests

```
npm start           # in one terminal
npm run test:e2e --prefix server  # in a second terminal
```

10.6 Live deployment

The same code is deployed on Vercel with a MongoDB Atlas database: <https://campuscoin-sable.vercel.app>. Every push to the `main` branch of the GitHub repository deploys automatically.

11 User Credentials

Accounts for every type of user.

These accounts are created automatically on first start and exist on the live site. Their passwords are fixed so that evaluators can always sign in; they cannot be changed, reset or locked.

Administrator

Sign in at `/admin/login`

Email: `admin@campuscoin.app`

Password: `Admin@12345`

Student (demo)

Sign in at `/login`

Email:

`student@campuscoin.app`

Password: `Student@12345`

Ayesha Khan, University
year 2

Student (second)

Sign in at `/login`

Email: `bilal@campuscoin.app`

Password: `Student@12345`

Bilal Ahmed, University
year 3

User type	What they can do
Student	Record, import and scan transactions; manage own categories, budgets and profile; view dashboard, calendar, reports, insights and tips; chat with Coin; receive alerts and emails
Administrator	View usage statistics; view, disable, reset or remove student accounts; manage default categories; publish announcements and tip templates

New student accounts can be created from the Register page at any time. A student account cannot open the admin panel even with the admin URL.

12 Acknowledgement of AI Tools

AI tools used, as required by the SRS.

As required by the SRS, the AI tools used during this project are acknowledged here.

Tool	How it was used
Claude Code (Anthropic)	Coding assistant used during development for writing and debugging code, design iterations, screenshots and testing, and help in drafting this report. All code and text were reviewed, and the design decisions are understood and can be explained.
Anthropic Claude API (optional feature)	Inside the application, when an API key is configured, it rewrites already-computed monthly figures in friendlier words and answers chat questions the built-in rules do not recognise. It never calculates any figure.
Tesseract.js	Open-source OCR engine used in the browser to read receipt photos.

Other resources: Microsoft Fluent Emoji (MIT licence) for category pictures, Simple Icons (CC0) for merchant logos, and design inspiration from finance dashboards on Dribbble. No ready-made website template was used.

Summary and possible extensions.

Campus Coin meets the functional and non-functional requirements of the SRS. A student can record money in seconds, see where it goes, set budgets that warn in time, and read advice that is built from their own history rather than general rules. The application is deployed publicly, works on phones and computers, and is covered by an automated test that exercises every requirement.

Possible future work

- Shared budgets for hostel roommates splitting rent and groceries.
- Reading SMS or email payment alerts from JazzCash and Easypaisa to add transactions automatically.
- An Urdu interface option.
- Savings goals for a target date (a laptop, a trip) with progress tracking.
- Installable progressive web app with offline entry.

Glossary

Term	Meaning
MERN	MongoDB, Express.js, React and Node.js - the four technologies the application is built with
API	Application Programming Interface - the /api routes the React client calls
JWT	JSON Web Token - a signed token that proves who is signed in
bcrypt	A slow, salted hashing method for storing passwords safely
OCR	Optical Character Recognition - reading text from a photo (used for receipts)
CSV	Comma-separated values - a simple spreadsheet file used for import and export
DFD	Data Flow Diagram - shows how data moves between processes, stores and people
ER diagram	Entity-Relationship diagram - shows the database entities and how they relate
SMTP	Simple Mail Transfer Protocol - how the server sends email (through Gmail)
Serverless	Hosting where the server code runs on demand (Vercel functions)

References

- Aptech Limited, *Campus Coin - Software Requirements Specification, Version 1.0*.
- React documentation - <https://react.dev>
- Node.js documentation - <https://nodejs.org/docs>
- Express 5 documentation - <https://expressjs.com>
- MongoDB and Mongoose documentation - <https://www.mongodb.com/docs>, <https://mongoosejs.com>
- Tesseract.js - <https://tesseract.projectnaptha.com>
- Vercel documentation - <https://vercel.com/docs>
- Microsoft Fluent Emoji (MIT) - <https://github.com/microsoft/fluentui-emoji>
- Simple Icons (CC0) - <https://simpleicons.org>